



Finanziato
dall'Unione europea
NextGenerationEU



MySQL

RDBMS



Cofinanziato
dall'Unione europea



INDICE argomenti

Cos'è un database

DBMS

Il Modello RELAZIONALE

Normalizzazione

Tipi di dato

Installazione MySQL

Accesso DBMS (DBA)

Accesso DBMS (user)

SQL

DDL

1 parte

2 parte

DCL

DML

Query

Integrità referenziale

Union e JOIN

Funzioni

Raggruppamenti

Windows Function (Funzioni finestra)

Viste

Sub Query

Backup/Restoring

Indici

Transaction

Triggers

Stored Function

Stored Procedures

Events

Cos'è un Database

Un database è una raccolta organizzata di dati archiviati in un formato facilmente accessibile.

- I **database** (o basi di dati) sono **collezioni di dati** tra loro correlati, utilizzate per rappresentare una porzione del mondo reale. Sono strutturati in modo tale da consentire la gestione efficiente dei dati, permettendo operazioni come **inserimento**, **aggiornamento**, **ricerca** e **cancellazione** delle informazioni.
- In informatica, il termine database indica una **struttura organizzata di dati**.
I **database** (o più brevemente, DB) sono archivi dove le applicazioni **memorizzano i dati** in modo **persistente**¹ per poterli successivamente leggere, modificare o eliminare.

¹) *In ambito informatico, la persistenza si riferisce alla capacità di un dato di essere conservato oltre la durata di esecuzione di un singolo programma, garantendo che i dati siano ancora disponibili in seguito.*

Dati e Informazione

- **Informazione:** dato elaborato con significato e contesto.

In senso generale, l'informazione è qualsiasi contenuto significativo che possiamo trasmettere, raccogliere o interpretare. Tuttavia, perché l'informazione diventi utile, deve essere organizzata e strutturata in una forma che ne consenta una facile elaborazione.

- **Dati:** rappresentazioni grezze di fatti, numeri o eventi.

Un dato è un'unità elementare di informazione, che può essere un numero, una parola, un'immagine, ecc.

Ad esempio, una stringa di caratteri come "Maria", oppure un numero come "30", sono dati grezzi. Da soli, però, non raccontano molto.

Per renderli significativi dobbiamo inserirli in un contesto strutturato, ad esempio: "Maria ha 30 anni."

Database file-server, client-server

database file-server

Sono semplici files, a cui possono facilmente accedere i programmi che li usano per inserire, visualizzare, modificare o cancellare i dati in essi contenuti.

- ◉ il sistema accede fisicamente al file;
- ◉ più il file è di grandi dimensioni maggiore il tempo di accesso;
- ◉ accesso contemporaneo da più utenti rallenta notevolmente il db;
 - *MS Access,*
 - *Filemaker*
 - ...;

database client-server

Rappresentano un servizio che mette a disposizione il software per interagire con i dati.

Viene gestito e mantenuto dai DBA (Database Administrator).

- *Microsoft SQL Server (RDBMS),*
- *Oracle (RDBMS),*
- *MySQL (RDBMS),*
- *DB2 (RDBMS),*
- *PostgreSql (ORDBMS object-relational database system),*
- *MongoDB (NoSQL Document Stores),*
- *Neo4j (NoSQL Graph),*
- ...;

client-server

Esempio di richiesta dati attraverso un form via https

CLIENT



*html
form*

WEBSERVER



*Php
Pyton
Java*

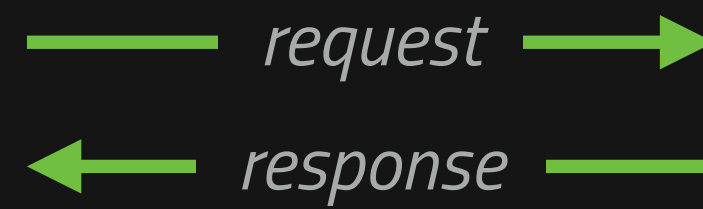
...

DATABASE

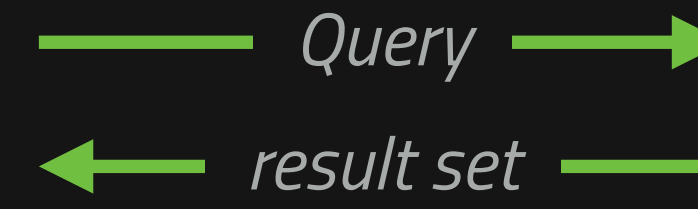


*MySQL
Dati*

< HTTP >



< SQL >



VANTAGGI DATABASE CLIENT-SERVER

1. I clients **non accedono fisicamente al file** sul database, inviano solamente la loro query al motore del database ed il server restituisce solamente i dati richiesti.
2. **Velocità**: al crescere delle dimensioni del database il tempo di una query rimane identico, perché attraverso la LAN viaggiano e continueranno a viaggiare solamente la richiesta (query) ed i dati restituiti, la dimensione del database diventa alla fine irrilevante per il client.
3. Il motore del database è in grado di gestire tutte le **connessioni simultanee** da parte degli utenti, ed utilizzare al meglio le prestazioni dell'hardware.
4. **Sicurezza**. Se su un sistema file-server potrebbe succedere che in determinate situazioni il file arrivi ad essere corrotto (termine tecnico), questo non deve potere succedere, mai e per nessuna ragione, su un sistema client-server.
5. La sicurezza *viene garantita anche grazie alle funzioni che i db client-server normalmente offrono*. Tutte le tabelle di un sistema gestionale aziendale sono tra loro collegate, la mancanza della gestione delle relazioni può portare a grossi problemi circa l' **integrità dei dati**.

Strutture Dati e Modelli di Database

Le strutture dati rappresentano il modo in cui le informazioni vengono organizzate e memorizzate per facilitarne il recupero e la gestione.

Ogni struttura ha caratteristiche specifiche che la rendono più o meno adatta a determinati scenari applicativi.

Nei database, le strutture dati vengono implementate attraverso diversi modelli di organizzazione, ognuno pensato per esigenze specifiche.

Modelli di database più diffusi

Relazionale (RDBMS - Relational Database Management System)

- ◉ Organizza i dati in tabelle (righe e colonne).
- ◉ Relazioni tra le tabelle tramite chiavi primarie ed esterne.

In questo modello, i dati sono organizzati in tabelle (o relazioni), che possono essere pensate come fogli di calcolo.

Ogni tabella ha delle colonne (campi) e delle righe (record), dove le righe rappresentano entità specifiche e le colonne descrivono attributi delle entità.

Esempi: *MySQL, PostgreSQL, Oracle, SQL Server...*

Time Series Database

Le serie temporali sono una struttura in cui i dati vengono organizzati in base a un intervallo di tempo. Vengono utilizzate per tracciare cambiamenti e osservare tendenze nel tempo.

- ◉ Ottimizzato per la memorizzazione di dati temporali.
- ◉ Formato: Timestamp + Valore.

Esempi: *InfluxDB*, *TimescaleDB*, *OpenTSDB*.

Esempio: sensori di temperatura che registrano la temperatura ogni ora:

```
{  
  timestamp: "2025-02-21T12:00:00Z",  
  temperatura: 22.5  
}
```

Database Colonnare

- Memorizza i dati per colonne anziché per righe.
- Vantaggioso per analisi su grandi volumi di dati.

Esempi: *Apache Cassandra, Google Bigtable, Amazon Redshift.*

Esempio: Analisi di vendite con colonne separate per ogni mese:

Prodotto | Gennaio | Febbraio | Marzo

NoSQL (Not Only SQL)

- ◉ Pensato per scalabilità e flessibilità.
- ◉ Tipologie:
 - ◉ Document-oriented: Archivia dati in documenti JSON o BSON (es. *MongoDB*, *CouchDB*).
 - ◉ Key-Value Store: Struttura chiave-valore (es. *Redis*, *DynamoDB*).
 - ◉ Wide-Column Store: Variante del database colonnare (es. *Apache HBase*, *ScyllaDB*).
 - ◉ Graph Database: Gestisce dati con nodi e connessioni (es. *Neo4j*).

Esempio: database documentale MongoDB che memorizza profili utente:

```
{  
  "nome": "Mario",  
  "cognome": "Rossi",  
  "email": "mario.rossi@example.com"  
}
```

DATABASE

uso dei database relazionali

Strumenti

- ◉ MySQL Community Server 8.4
- ◉ Terminale/Shell
 - ambiente a riga di comando per interagire con il database tramite istruzioni SQL
- ◉ MySQLWorkbench: ambiente grafico per interagire con il database
- ◉ Visual Studio Code o altro editor (Sublime Text, Atom, Brackets...)
- ◉ Documentazione ufficiale MySQL:
 - <https://dev.mysql.com/doc/refman/8.4/en>

DBMS

database management system

- ◉ È il software per la creazione e la manipolazione di un database.
È un **software di tipo server** (client-server) avente il compito di gestire uno o più database; questo vuol dire che il DBMS deve intervenire, **in qualità di intermediario**, in ogni operazione svolta sui database dai software che ne fanno utilizzo.
- ◉ Definisce gli utenti e gli amministratori di un database
- ◉ Fornisce **meccanismi di sicurezza, protezione** e controllo dell'**integrità dei dati**

RDBMS

relational database management system

- MySQL è un software appartenente alla famiglia dei DBMS. All'interno di questo gruppo di software è possibile identificare dei sotto-insiemi più specifici tra cui, ad esempio, quello dei **DBMS NoSQL** (MongoDB) e quello dei **RDBMS** a cui appartiene tra gli altri, appunto, MySQL.
- Gli RDBMS non sono altro che dei sistemi di gestione delle banche dati che operano in aderenza alla teoria relazionale secondo la quale il sistema deve operare sui dati mediante relazioni tra le diverse tabelle in cui questi vengono suddivisi e ordinati.
- Nel modello relazionale, infatti, i dati all'interno di un database *sono organizzati in differenti tabelle le quali sono in relazione tra loro.*

Storage Engine

Gli **storage engine** rappresentano delle librerie che determinano il modo in cui i dati di una tabella saranno salvati su disco.

Ciò sarà determinante per valutare le prestazioni, l'affidabilità, le funzionalità offerte dalla tabella stessa, rendendola più o meno adatta a particolari utilizzi.

In pratica, scegliere un particolare storage engine significa scegliere il modo in cui i dati vengono gestiti.

MyISAM

Si tratta di un motore di memorizzazione veloce. Non supporta le transazioni. Non utilizza meccanismi di integrità referenziale.

- ◉ Adatto per le **ricerche full-text**;
- ◉ È **più veloce** poiché non è necessario tenere conto delle varie relazioni tra le tabelle;
- ◉ esegue il **lock sull'intera tabella**;
- ◉ ottimo se le tabelle vengono utilizzate principalmente in fase di lettura oppure se il **database è relativamente poco complesso**.

A partire dalla versione 5.5 di MySQL, *InnoDB è lo Storage Engine di default*. Prima era MyISAM.

InnoDB

Lo scopo di InnoDB è quello di associare maggiore sicurezza (intesa soprattutto come consistenza ed integrità dei dati) a performance elevate. Funzionalità peculiari:

- ◉ **Transizioni**: per transazione si intende la possibilità di un DBMS di svolgere più operazioni di modifica dei dati, facendo sì che i risultati diventino persistenti nel database solo in caso di successo di ogni singola operazione. In caso contrario, verranno annullate tutte le modifiche apportate;
- ◉ **Integrità referenziale**: conferiscono la possibilità di creare una relazione logica tra i dati di due tabelle, in modo da impedire modifiche all'una che renderebbero inconsistenti i dati dell'altra;
- ◉ esegue il **lock a livello di riga**;
- ◉ *Ricerche full-text* a partire da MySQL 5.6.

Charset

I **character set** (insiemi di caratteri) sono i diversi sistemi attraverso i quali i caratteri alfanumerici, i segni di punteggiatura e tutti i simboli rappresentabili su un computer vengono memorizzati in un valore binario. In ogni set di caratteri, ad un valore binario corrisponde un carattere ben preciso.

Con MySQL, a partire dalla versione 4.1, possiamo gestire i set di caratteri a livello di server, database, tabella e singola colonna, nonché di client e di connessione.

Ad ogni set di caratteri sono associate una o più **collation**, che rappresentano i modi possibili di confrontare le stringhe di caratteri facenti parte di quel character set.

Esempio: una determinata tabella utilizza il *utf8* (quello maggiormente usato in Europa Occidentale) e la *utf8_general_ci*.

Tale collation è multilingue e "case insensitive" (*_ci*), cioè non tiene conto della differenza fra maiuscole e minuscole nell'ordinare o confrontare le stringhe.

Supponiamo di avere un alfabeto di quattro lettere:

A, B, a, b.

Assegnamo ad ogni lettera un numero: $A = 0$, $B = 1$, $a = 2$, $b = 3$.

La lettera A è un *simbolo*, il numero 0 è la *codifica* per A, e la combinazione di tutte e quattro le lettere e la loro codifica è il **character set**.

Supponiamo che vogliamo confrontare due valori di stringa, A e B. Il modo più semplice per farlo è quello di guardare le codifiche: 0 per A e 1 per B.

Poiché 0 è minore di 1, diciamo A è inferiore a B.

Quello che abbiamo appena fatto è applicare un metodo di confronto per il nostro set di caratteri.

La **collation** è un insieme di regole (una sola regola in questo caso): "Confronta le codifiche".

IL MODELLO RELAZIONALE

- La **tabella** è la struttura dati fondamentale di un database relazionale;
- Con le tabelle si rappresentano le **entità** e le **relazioni** del modello concettuale*;
- La tabella è composta da campi (colonne o **attributi**) e da record (righe o **tuple**);
- Ogni *campo* rappresenta un **attributo** dell'entità/ relazione;
- Per ogni campo viene individuato un suo **dominio** (*tipo di dati*): alfanumerico, numerico, data...
- Ogni *record* rappresenta una *istanza* (o occorrenza o *tupla*) dell'entità/relazione;
- Garantisce l'**integrità referenziale**;

***Modello concettuale**: trasformazione di **specifiche** in linguaggio naturale (che definiscono la realtà descritta dal DB) in uno schema grafico chiamato **Diagramma E-R** che utilizza due concetti fondamentali: *Entità* e *Associazione/Relazione*.

Progettazione

Modello Concettuale

Definizione: rappresenta i dati e le loro relazioni a un livello astratto, senza preoccuparsi di dettagli tecnici o implementativi.

Obiettivo: descrivere la struttura dei dati in modo comprensibile per utenti e analisti, senza vincoli tecnologici.

Strumento tipico: diagrammi Entity-Relationship (ER) o UML class diagrams (Unified modeling language).

Indipendenza dal DBMS: Questo modello non dipende dal database relazionale, NoSQL o altro.

Modello Logico

Definizione: traduce il modello concettuale in una struttura più dettagliata e aderente alle regole di un particolare tipo di database (es. relazionale).

Obiettivo: definire tabelle, attributi, chiavi primarie e chiavi esterne, mantenendo ancora un livello di astrazione dalla tecnologia specifica.

Strumento tipico: Schema relazionale (tabelle, colonne, vincoli).

Dipendenza dal tipo di database: nel caso del modello relazionale, si definiscono relazioni tra tabelle con chiavi primarie e chiavi esterne.

Modello Fisico

Definizione: rappresenta la struttura effettiva del database come verrà implementata su un determinato DBMS.

Obiettivo: ottimizzare il database per le prestazioni, definendo dettagli come tipi di dati specifici, indici, partizionamento, ecc.

Strumento tipico: DDL (Data Definition Language) SQL per creare tabelle, vincoli, indici.

Dipendenza dal DBMS: strettamente legato a un DBMS specifico (es. MySQL, PostgreSQL, Oracle).

Fasi della progettazione

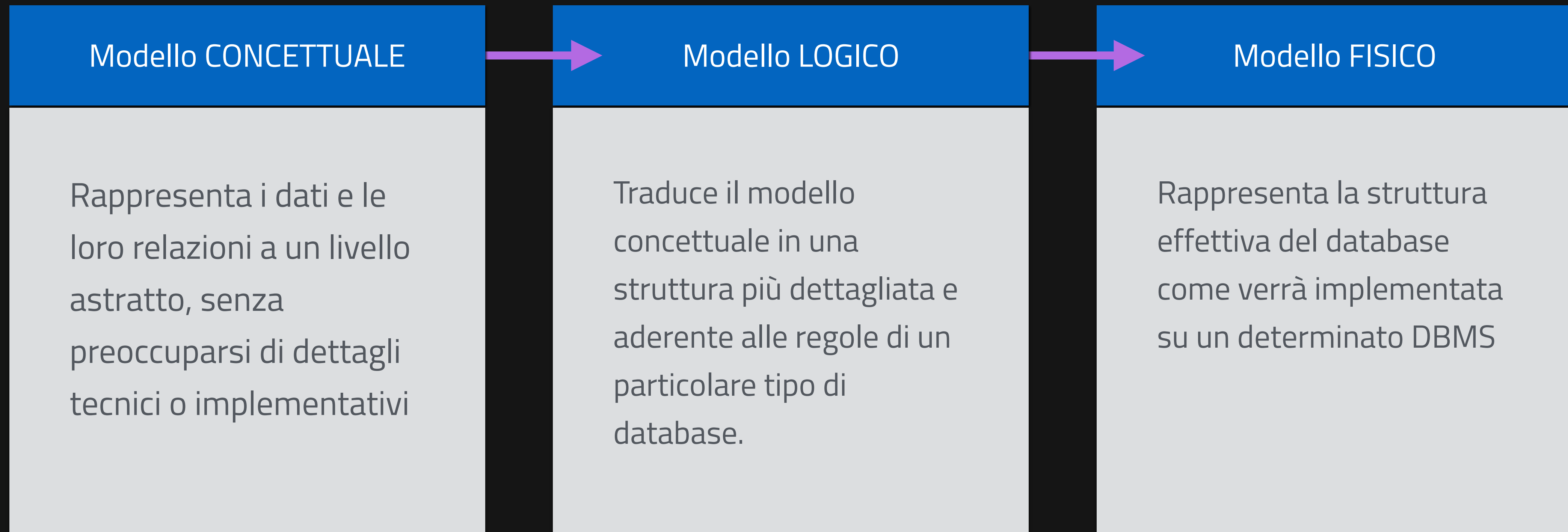


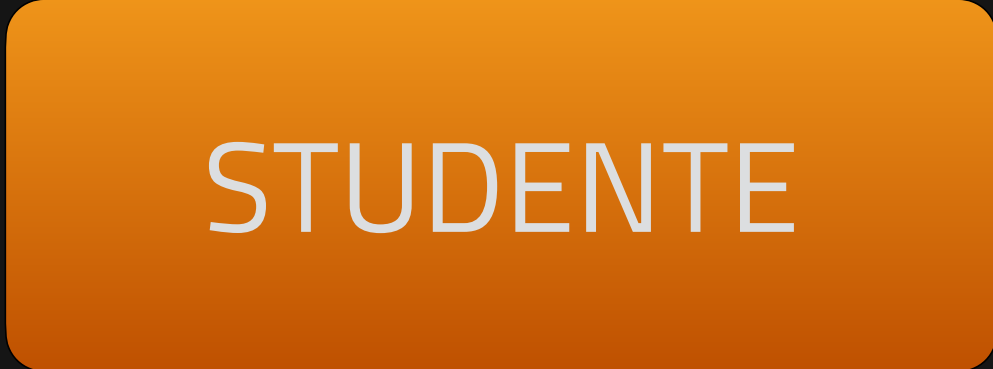
Diagramma E-R, simboli

Entità

Concetto fondamentale, generale, per la realtà che si sta modellando.

Rappresenta *classi di oggetti* (fatti, cose, persone, ...) che hanno *proprietà comuni* ed *esistenza autonoma* ai fini dell'applicazione di interesse.

- ◉ Identificata da un rettangolo

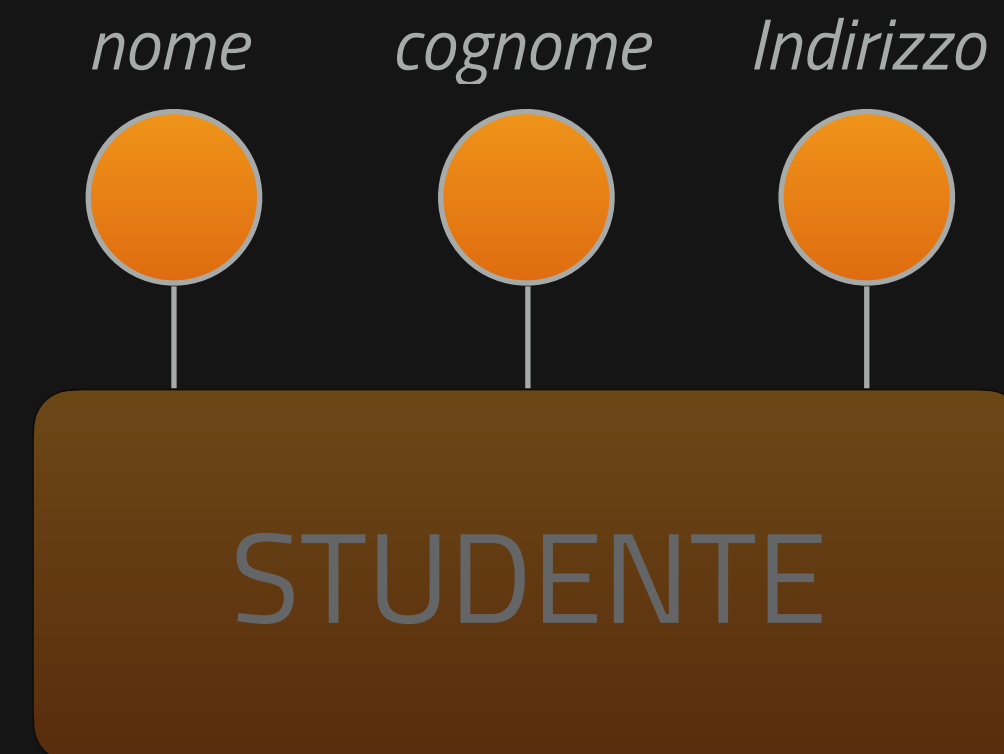


STUDENTE

Attributi:

Caratteristiche specifiche di un'entità, utili (o necessarie) nella realtà da modellare

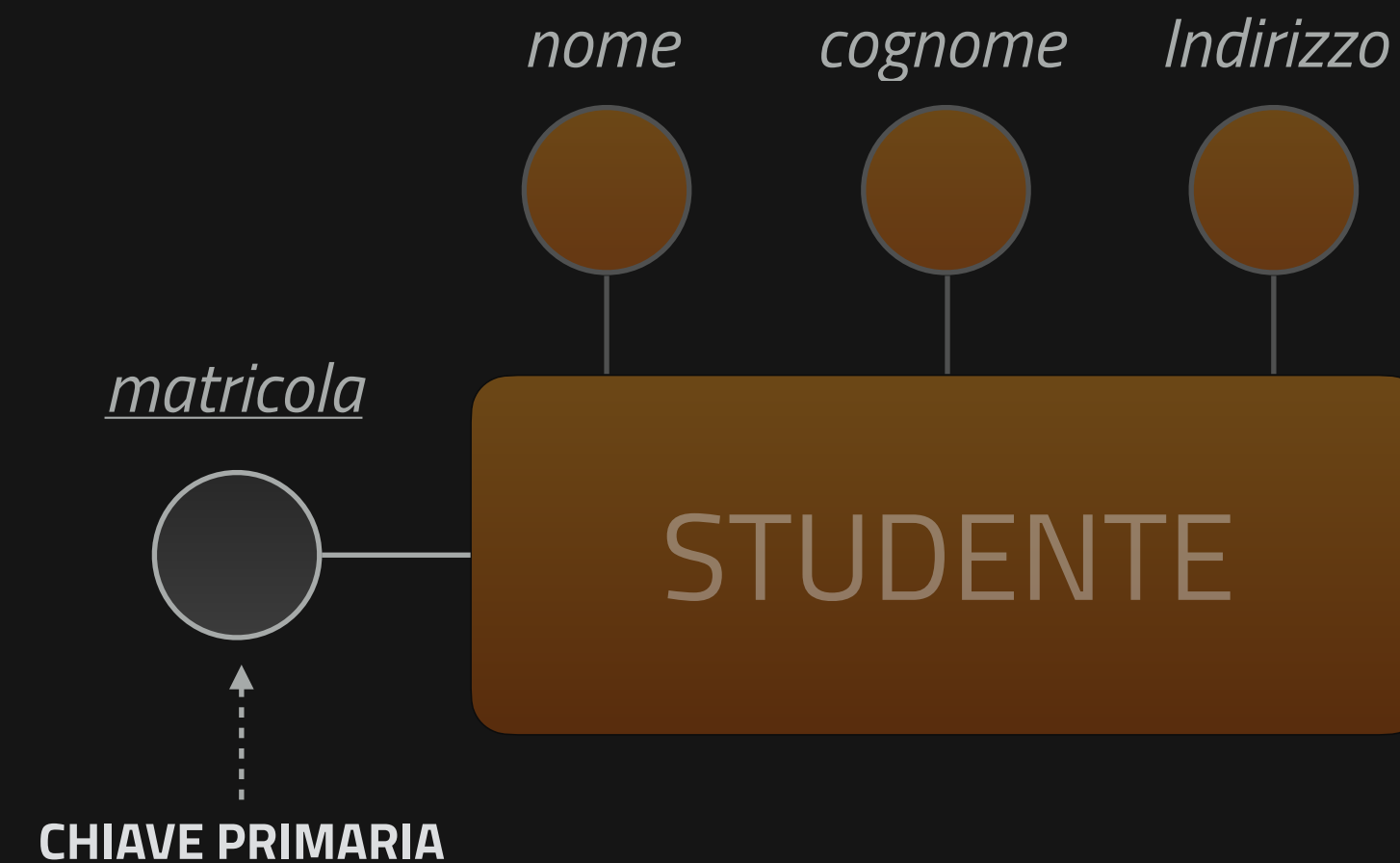
- identificata da un cerchio collegato all'entità



L'insieme di attributi che garantisce **l'univocità** delle istanze di un'entità è detta: **Chiave Primaria**.

È indicata come: **PRIMARY KEY** o **PK**

Identificata graficamente con un *cerchio pieno* collegato all'entità e relativo *nome attributo sottolineato*



Caratteristiche

- L'insieme dei campi i cui valori identificano univocamente un record all'interno di una tabella è detto Chiave Primaria. Quando la *chiave primaria* è composta da un solo campo, si parla di *campo chiave*.
- Quando non è possibile trovare un campo chiave tra gli attributi di una entità, si definisce un campo univoco di tipo numerico che si auto-incrementa (contatore): ID (identifier).

Esempi di campo chiave: *matricola*, *codice fiscale*, etc.

Istanze di un'Entità

Specifici dati, oggetti appartenenti ad un'entità

- non sono rappresentate nel Diagramma E-R ma si intendono contenute in ogni entità:
- Carlo Rossi, via Verdi* è un'istanza dell'**entità** *ALUNNO* (**attributi**: *Nome, Cognome, Indirizzo*)



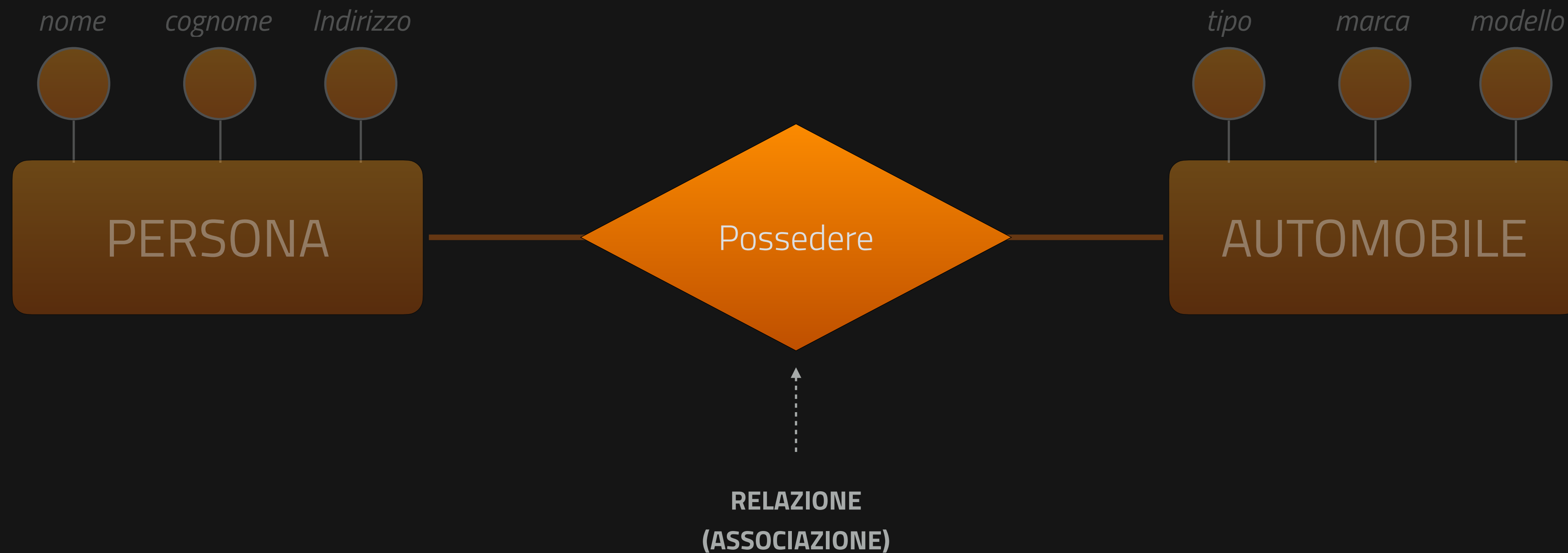
Possiamo considerare le entità come insiemi all'interno dei quali sono contenuti *oggetti* (le istanze) ciascuno con specifiche *caratteristiche* (valore degli attributi).



Relazioni (Associazioni)

Collegamenti logici che uniscono due o più entità nella realtà descritta dal database

- identificata da un rombo collegato alle due entità:



Cinema/teatro

Relazione 1,1 (uno a uno)



- Uno spettatore *occupa* un singolo posto
- Ogni singolo posto può *essere occupato* solo da uno spettatore

Liceo/Scuola superiore

Relazione 1,N (uno a molti)



- Ad ogni classe *appartiene* più di un alunno
- Un alunno *appartiene* ad una singola classe

Università

Relazione N,N (molti a molti)



- Uno studente *frequenta* più corsi
- Ogni corso è *frequentato* da molti studenti

Cardinalità delle relazioni

La relazione R che lega due entità **E1** ed **E2** può essere classificata in base alla sua **cardinalità** (quante istanze delle due entità sono coinvolte nella relazione):

- ◉ $1,1$ (**uno a uno**) se ad un elemento di **E1** può corrispondere un solo elemento di **E2**
- ◉ $1,N$ (**uno a molti**) se ad un elemento di **E1** possono corrispondere più di un elemento di **E2**, ad un elemento di **E2** può corrispondere un solo elemento di **E1**
- ◉ N,N (**molti a molti**) se ad ogni elemento di **E1** possono corrispondere molti elementi di **E2** e viceversa

Esempi



Esempio

Database per una applicazione web che gestisce l'acquisto/iscrizione dei/ai corsi.

Disegnare una base dati per la gestione dell'acquisto di corsi offerti da una piattaforma web.

Gli utenti/studenti devono essere registrati sulla piattaforma, per registrarsi occorre nome, cognome ed email. Viene memorizzata anche la data di registrazione.

Gli studenti/utenti possono acquistare molti corsi. I corsi sono quindi a pagamento. Viene memorizzata anche la data di acquisto.

Il prezzo del corso può variare nel tempo.

I corsi si riferiscono a svariate materie quali: Java, Base di programmazione, Html, CSS, CMS, Javascript, React, PHP...

Ogni corso viene tenuto da un docente. Un docente può insegnare in molti corsi.

Modello Concettuale

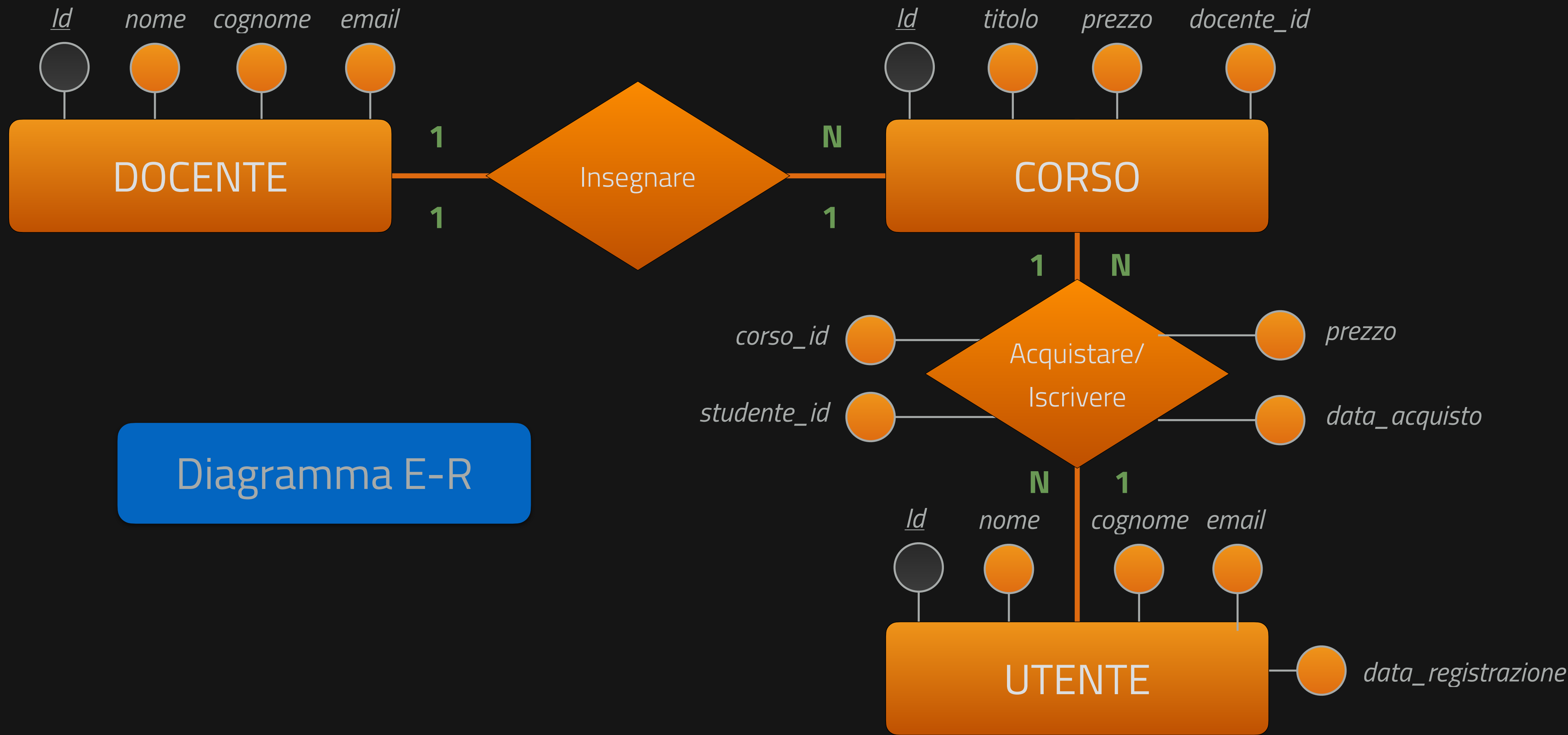


Diagramma E-R

Modello logico

Simbologia delle relazioni



Rappresentazione dell'informazione in tabelle nel database secondo il modello relazionale dell'esempio relativo alla piattaforma corsi:

UTENTI

● id	cognome	nome	email
1	Esposito	Giuseppe	esposito.g@icloud.com
2	Pautasso	Carlo	cpautasso@gmail.com
3	Piras	Efisio	efpiras@gmail.com
4	Conti	Renzo	conti.renzo@icloud.com
5	Rigon	Paolo	prigon@gmail.com

ISCRIZIONI

corso_id	studente_id	prezzo	data
1	1	180.00	22/9/2022
1	2	180.00	23/9/2022
3	1	300.00	25/9/2022
2	3	120.00	28/9/2022

DOCENTI

● id	cognome	nome	email
1	Rossi	Mario	rossi.mario@icloud.com
2	Verdi	Paola	verdi.paola@gmail.com
3	Bruni	Marco	bruni.marco@gmail.com
4	Bianchi	Daniele	bianchi.daniele@icloud.com

CORSI

● id	titolo	prezzo	docente_id
1	Php	200.00	4
2	Javascript	120.00	2
3	Java	300.00	3

Legame logico tra tabelle

CORSI

Chiave Esterna

Campo Chiave

● id	titolo	prezzo	docente_id	● id	cognome	nome	email
1	Php	200.00	4	1	Rossi	Mario	rossi.mario@icloud.com
2	Javascript	120.00	2	2	Verdi	Paola	verdi.paola@gmail.com
3	Java	300.00	3	3	Bruni	Marco	bruni.marco@gmail.com
				4	Bianchi	Daniele	bianchi.daniele@icloud.com

Join tra tabelle

● id	titolo	prezzo	docente_id	● id	cognome	nome	email
1	Php	200.00	4	4	Bianchi	Daniele	bianchi.daniele@icloud.com
2	Javascript	120.00	2	2	Verdi	Paola	verdi.paola@gmail.com
3	Java	300.00	3	3	Bruni	Marco	bruni.marco@gmail.com

Normalizzazione del database

Regole da rispettare nel definire le tabelle

La prima forma normale (1NF)

Si dice che una database è in 1NF (prima forma normale) se per ogni tabella/relazione contenuta nella base dati:

- *tutte le tuple della relazione hanno lo stesso numero di attributi*
- *non presenta gruppi di attributi che si ripetono* (ossia ciascun attributo è definito su un dominio con valori atomici)
- *tutti i valori di un attributo sono dello stesso tipo* (appartengono allo stesso dominio)
- *esiste una chiave primaria* (ossia esiste un attributo o un insieme di attributi che identificano in modo univoco ogni tupla della relazione)
- *l'ordine delle righe è irrilevante* (non è portatore di informazioni)

Facciamo un esempio di una tabella che, seppur munita di una chiave primaria, non può essere considerata in forma normale:

● Codice Fiscale	Nome	Dettagli
LBRRSS79Y12T344A	Alberto	età: 30; professione: Impiegato
GNNBNCT84A11L61B	Gianni	età: 24; professione: Studente

La tabella qui sopra NON è in 1NF in quanto, pur avendo una chiave primaria, presenta un attributo (dettagli) che non contiene dati in forma atomica, ma un gruppo di attributi.

● Codice Fiscale	Nome	Dettagli
LBRRSS79Y12T344A	Alberto	età: 30; professione: Impiegato
GNNBNCT84A11L611B	Gianni	età: 24; professione: Studente

Modifichiamo la tabella aggiungendo gli opportuni attributi:

● Codice Fiscale	Nome	eta	professione
LBRRSS79Y12T344A	Alberto	30	Impiegato
GNNBNCT84A11L611B	Gianni	24	Studente

La seconda forma normale (2NF)

Perché una base dati possa essere in 2NF è necessario che:

- si trovi già in 1NF;
- tutti gli *attributi non chiave* dipendano dall'intera chiave primaria (e non solo da una parte di essa).

Per fare un esempio si supponga di avere a che fare con il database di una scuola con una chiave primaria composta dai campi "Codice Matricola" e "Codice Esame":

● Codice Matricola	● Codice Esame	Nome Matricola	Voto Esame
1234	M01	Rossi Alberto	6
1234	L02	Rossi Alberto	7
1235	L02	Verdi Mario	8

Il database qui sopra si trova in 1NF ma non in 2NF

Perché il campo "Nome Matricola" non dipende dall'intera chiave ma solo da una parte di essa ("Codice Matricola").

• Codice Matricola	• Codice Esame	Nome Matricola	Voto Esame
1234	M01	Rossi Alberto	6
1234	L02	Rossi Alberto	7
1235	L02	Verdi Mario	8

Per rendere il nostro database in 2NF dovremo scomporlo in due tabelle:

• Codice Matricola	• Codice Esame	Voto Esame	• Codice Matricola	Nome Matricola
1234	M01	6	1234	Rossi Alberto
1234	L02	7	1235	Verdi Mario
1235	L02	8		

La terza forma normale (3NF)

Un database è in 3NF se:

- è già in 2NF (e quindi, necessariamente, anche 1NF);
- tutti gli attributi non chiave dipendono direttamente dalla chiave, quindi *non ci sono attributi "non chiave" che dipendono da altri attributi "non chiave"*.

Supponiamo di avere una base dati di una palestra in cui il codice fiscale dell'iscritto al corso frequentato è associato all'insegnante di riferimento.

Si supponga che il nostro DB abbia un'unica chiave primaria ("Codice Fiscale") e sia così strutturato:

• Codice Fiscale	Codice Corso	Insegnante
LBRRSS79Y12T344A	BB01	Marco
GNNBNCT84A11L61B	BB01	Marco
LBRMNN79E64A112A	BB01	Marco
GLSTMT59U66P109B	AE02	Federica

Il nostro database non è in 3NF in quanto il campo "insegnante" non dipende dalla chiave primaria ma dal campo "Codice Corso" (che non è chiave).

• Codice Fiscale	Codice Corso	Insegnante
LBRRSS79Y12T344A	BB01	Marco
GNNBNCT84A11L611B	BB01	Marco
LBRMNN79E64A112A	BB01	Marco
GLSTMT59U66P109B	AE02	Federica

Per normalizzare il nostro DB in 3NF dovremo scomporlo in due tabelle:

• Codice Fiscale	Codice Corso
LBRRSS79Y12T344A	BB01
GNNBNCT84A11L611B	BB01
LBRMNN79E64A112A	BB01
GLSTMT59U66P109B	AE02

• Codice Corso	Insegnante
BB01	Marco
AE02	Federica

Tipi di dato

In una tabella MySQL per ciascuna colonna possiamo definire diversi tipi di dato (dominio):

- ◉ **Numerics** (numeri interi e a virgola mobile)
- ◉ **String** (stringa)
- ◉ **Date , Time** (data e ora)
- ◉ **JSON**

Dati numerici: interi

Tipo	Intervallo di valori	Solo se positivi (UNSIGNED)
BIT(M)	Da 1 a 64	/
TINYINT (1 byte)	da -128 a +127	da 0 a 255
SMALLINT (2 byte)	da -32 768 a +32 767	da 0 a 65 535
MEDIUMINT (3 byte)	da -8 388 608 a +8 388 607	da 0 a 16 777 215
INT (4 byte)	da -2 147 483 648 a +2 147 483 647	da 0 a 4 294 967 295
BIGINT (8 byte)	da -9 223 372 036 854 775 808 a +9 223 372 036 854 775 807	da 0 a 18 446 744 073 709 550 615

E' importante precisare che se all'interno di un campo di tipo numerico si cerca di inserire un valore maggiore di quanto ammesso dal tipo prescelto, MySQL produrrà un errore.

Tipi di numeri in virgola mobile (valore approssimativo)

Tipo	Tipo (sintassi)	Tipo (sintassi deprecata)*	spazio
FLOAT	FLOAT[(p)], con p compreso tra 0 e 23	FLOAT, FLOAT(M,D)*	4byte
DOUBLE	FLOAT[(p)], con p compreso tra 24 e 53	DOUBLE, DOUBLE(M,D)*	8byte
DECIMAL	DECIMAL(M,D)		Dipende da M*

Nel caso di FLOAT, puoi specificare opzionalmente la precisione, usando una sintassi tipo FLOAT(p), dove "p" è la precisione in bit.

Se la precisione è tra 0 e 23, la colonna sarà considerata di tipo FLOAT e userà 4 byte.

Se la precisione è tra 24 e 53, sarà considerata di tipo DOUBLE e userà 8 byte.

MySQL permette anche una sintassi non standard, del tipo FLOAT(M,D) (o DOUBLE(M,D)).

M è il numero totale di cifre che puoi memorizzare (prima e dopo il punto decimale).

D è il numero di cifre dopo il punto decimale.

Per esempio, se dichiari una colonna come FLOAT(7,4), puoi memorizzare fino a 7 cifre in totale, con 4 cifre dopo il punto decimale.

Quindi i numeri vanno da -999.9999 a 999.9999.

* <https://dev.mysql.com/doc/refman/8.4/en/precision-math-decimal-characteristics.html>

Dati stringa

Tipo e lunghezza
massima consentita

```
nome VARCHAR(20)
codiceFiscale CHAR(16)
titolo TINYTEXT
messaggio TEXT
```

Tipo	Lunghezza massima
CHAR(n)	255 caratteri
VARCHAR(n)	65.535 byte
BINARY(b)	255 byte
VARBINARY(b)	65.535 byte
TINYTEXT	256 caratteri
TINYBLOB	255 byte
TEXT	65.535 byte
BLOB	65,535 byte
MEDIUMTEXT	16.777.215 caratteri
MEDIUMBLOB	16,777,215 byte
LONGTEXT	4.294.967.295 caratteri
LOB	4.294.967.295 byte
ENUM('value1','value2',...)	2byte (65.535 valori distinti)
SET('value1','value2',...)	64 valori distinti

I tipi **CHAR** e **VARCHAR** sono sicuramente i tipi più utilizzati.

La differenza tra questi due tipi è data dal fatto che CHAR ha *lunghezza fissa*, VARCHAR ha *lunghezza variabile*.

Questo significa che in una colonna **CHAR**(10) tutti i valori memorizzati occuperanno lo spazio massimo anche se costituiti da 3 soli caratteri.

I tipi **TEXT** e **BLOB** (Binary Large Object) consentono di memorizzare grandi quantità di dati:

- ◉ TEXT è utilizzato per dati di tipo testuale,
- ◉ BLOB è utilizzato per ospitare dati binary (ad esempio il sorgente di un'immagine)

BINARY e VARBINARY

I tipi **BINARY** e **VARBINARY** sono simili a CHAR e VARCHAR, tranne per il fatto che *memorizzano stringhe binarie* anziché stringhe non binarie: memorizzano stringhe di byte anziché stringhe di caratteri.

Per questi campi il *set di caratteri* e la *collation*, il *confronto* e l'*ordinamento* si basano sui valori numerici dei byte memorizzati.

TEXT vs VARCHAR()

TEXT

- ◉ dimensione massima fissa di 65535 caratteri (non è possibile limitare la dimensione massima)
- ◉ prende $2 + c$ byte di spazio su disco, dove c è la lunghezza della stringa memorizzata.
- ◉ indice: può essere indicizzato solo con un indice: prefix index.

VARCHAR (M)

- ◉ dimensione massima variabile di caratteri M
- ◉ M deve essere compreso tra 1 e 65535
- ◉ prende $1 + c$ byte (per $M \leq 255$) o $2 + c$ (per $256 \leq M \leq 65535$) byte di spazio su disco dove c è la lunghezza della stringa memorizzata
- ◉ può essere parte di un indice

Se è necessario memorizzare stringhe più lunghe di circa 64 KB, utilizzare MEDIUMTEXT o LONGTEXT.

VARCHAR non supporta la memorizzazione di valori così grandi.

Tipi ENUM e SET

I tipi ENUM e SET sono un tipo di dato di testo in cui le colonne possono avere solo dei valori predefiniti.

ENUM: Tipo di dato ENUMerazione.

Contiene un insieme di valori prefissati tra cui scegliere: **si può inserire solamente uno dei valori previsti.**

I valori sono inseriti tra parentesi(elenco separato da virgola) dopo la dichiarazione ENUM.

```
genere ENUM( 'F', 'M', 'NB' )
```

La colonna genere accetterà solamente i valori F , M o NB. Se proviamo a mettere un valore diverso con il comando INSERT, MYSQL restituirà errore.

SET: è una estensione di ENUM.

```
interessi SET( 'a', 'b', 'c', 'd' )
```

Come per ENUM i valori sono fissi e disposti dopo la dichiarazione SET; tuttavia, le colonne SET possono assumere più di un valore tra quelli previsti.

DateTime

Tali tipi di dati sono molto utili quando si ha a che fare con informazioni riguardanti la data e l'orario.
Di seguito una tabella riepilogativa

Tipo	Formato	Intervallo
DATETIME	YYYY-MM-DD HH:MM:SS	'1000-01-01 00:00:00' a '9999-12-31 23:59:59'
DATE	YYYY-MM-DD	1000-01-01' a '9999-12-31'
TIME	HH:MM:SS	-838:59:59' a '838:59:59
YEAR	YYYY	un anno compreso fra 1901 e 2155, oppure 0000.
TIMESTAMP	YYYY-MM-DD HH:MM:SS	'1970-01-01 00:00:01'UTC' a '2038-01-19 03:14:07' UTC'

I campi di tipo DATETIME contengono sia la data che l'orario. I valori all'interno di questi campi possono essere inseriti sia sotto forma di stringhe che di numeri.

MySQL interpreta i valori dell'anno a 2 cifre utilizzando queste regole

- I valori dell'anno nell'intervallo 00-69 diventano 2000-2069.
- I valori dell'anno nell'intervallo 70-99diventano 1970-1999.

Sia *DATETIME* sia *TIMESTAMP* possono memorizzare in automatico la data.

Per ottenere ciò in fase di definizione del campo bisogna impostare il valore di *default di memorizzazione* (es):

```
ins TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
data DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```


DATETIME o TIMESTAMP?

TIMESTAMP

Il tipo TIMESTAMP memorizza data e ora insieme al fuso orario del server.

Quando memorizzi un valore in una colonna TIMESTAMP, MySQL lo converte automaticamente nel fuso orario UTC (Coordinated Universal Time) e lo memorizza in quel formato. Quando lo leggi, il valore viene riconvertito nel fuso orario del server o in quello del client (se configurato).

Questo rende il TIMESTAMP utile quando devi mostrare gli stessi dati di data e ora a utenti in fusi orari diversi, poiché MySQL gestisce automaticamente la conversione da e verso UTC.

TIMESTAMP copre solo il range di date da 1970-01-01 00:00:01 UTC fino a 2038-01-19 03:14:07 UTC.

Se hai bisogno di memorizzare date al di fuori di questo intervallo, non è adatto.

DATETIME o TIMESTAMP?

DATETIME

Il tipo DATETIME memorizza la data e l'ora così come sono, senza conversioni di fuso orario. Il valore viene memorizzato esattamente come viene inserito, senza alcuna considerazione del fuso orario del server o del client.

Questo significa che DATETIME è utile quando vuoi che la data e l'ora restino fisse, indipendentemente dal fuso orario dell'utente o del server.

È una buona scelta se i dati di data e ora devono essere gli stessi per tutti, ovunque si trovino.

DATETIME ha un range molto più ampio, da 1000-01-01 00:00:00 a 9999-12-31 23:59:59, quindi può essere usato per date molto più lontane nel passato o nel futuro rispetto a TIMESTAMP.

JSON

MySQL supporta JSON nativo come tipo di dati per gli oggetti nella notazione JSON.

Rende facile l'archiviazione, l'interrogazione e il recupero di documenti di tipo JSON piuttosto che archiviarli come stringhe di testo o BLOB binari (vedi MariaDB).

Per fare ciò mette a disposizione una serie di funzioni*

Sintassi per la definizione di un attributo di tipo JSON

```
columnName JSON
```

* <https://dev.mysql.com/doc/refman/8.0/en/json-function-reference.html>

Attributi

Per i campi numerici

Si tratta di un'istruzione ulteriore che viene passata al DBMS in fase di creazione (o modifica) di una tabella.

- **AUTO_INCREMENT** - Aumenta automaticamente il valore di una colonna aggiungendo 1 al valore massimo già registrato nella tabella. E' opportuno utilizzarlo in combinazione con NOT NULL. Può essere applicato a tutti i tipi numerici interi.
- **UNSIGNED*** - un campo contrassegnato con UNSIGNED non accetta valori negativi.
- **ZEROFILL**** - viene utilizzato per memorizzare i numeri con degli zeri davanti nel caso in cui la lunghezza sia inferiore a quella massima prevista.

Se per esempio si dichiara un campo INT ZEROFILL e si desidera memorizzare il valore "1234", esso verrà visualizzato come "00000001234" con M(11) - sempre UNSIGNED.

*A partire da MySQL 8.0.17 è deprecato per FLOAT, DOUBLE e DECIMAL; **A partire da MySQL 8.0.17 è deprecato; [vedi documentazione mysql](#).

AUTO_INCREMENT (sequence)

In MySQL, una *sequenza* è un elenco di interi generati nell'ordine crescente, vale a dire 1,2,3...

Impostare l'attributo AUTO_INCREMENT in una colonna, tipicamente una colonna chiave primaria, crea automaticamente una sequenza in MySQL.

Le seguenti regole vengono applicate quando si utilizza l'attributo AUTO_INCREMENT:

- ◉ Ciascuna tabella ha **solo una colonna** AUTO_INCREMENT il cui tipo di dati è **"intero"**.
- ◉ La colonna AUTO_INCREMENT **deve essere indicizzata**, il che significa che può essere:
 - PRIMARY KEY o UNIQUE.
- ◉ La colonna AUTO_INCREMENT deve avere un vincolo NOT NULL.

Quando si imposta l'attributo AUTO_INCREMENT in una colonna, **MySQL aggiunge automaticamente il vincolo NOT NULL** alla colonna implicitamente.

Attributi per i campi di tipo stringa

BINARY:

- ◉ L'unico vincolo/opzione che può essere utilizzato per i campi destinati ad ospitare dati stringa è **BINARY** il quale può essere utilizzato con **CHAR** o **VARCHAR** qualora questi campi siano destinati ad ospitare dati binari (pur non rendendosi necessario utilizzare un campo della famiglia BLOB)
- ◉ **CHAR(n) BINARY** e **VARCHAR(n) BINARY** differiscono dal *tipo di dato* **BINARY** e **VAR BINARY** per la codifica e la collation utilizzata*

* <https://dev.mysql.com/doc/refman/8.0/en/binary-varbinary.html>

Attributi universali (sia per campi numerici sia per quelli testuali)

Possono essere utilizzati tanto con campi numerici quanto con campi di tipo stringa.

- ◉ **DEFAULT** – Può essere utilizzato con tutti i tipi di dati ad eccezione di TEXT e BLOB. Serve per indicare un valore di default per il campo qualora questo venga lasciato vuoto.
- ◉ **NULL / NOT NULL** – Può essere utilizzato con tutti i tipi di campi e serve per definire se un dato campo può avere un valore NULL oppure no.

Vincoli

- ◉ **CHECK** (expression) – consente di imporre un vincolo al dato da inserire.
- ◉ **FOREIGN KEY** – consente di imporre un vincolo riferito alla chiave esterna.

Attributi/Indici

- ◉ **UNIQUE** – Con UNIQUE si imposta una regola di unicità, questo significa che nessun dato contenuto nella colonna può essere ripetuto: ogni dato deve, quindi, essere unico e se si cerca di inserire un dato duplicato si riceve un errore. Può essere nullo.
- ◉ **PRIMARY KEY** – Può essere utilizzato con tutti i tipi di dati (numerici e stringa) ed è una sorta di variante di UNIQUE che consente di creare un indice primario sulla tabella (campo chiave).
- ◉ **INDEX** [KEY] (*colonne*) – E' utilizzato per creare un'indice nella tabella ai fini di migliorare le performances di accesso ai dati.

INDEX

Indici

- Servono ad ottimizzare le performance del database.
- Un indice è una struttura dati ausiliaria che consente di recuperare più velocemente i dati di una tabella, evitandone la lettura dell'intero contenuto (full table scan), tramite una selezione più mirata.
- devono essere usati consapevolmente per non ottenere l'effetto contrario ovvero rallentare il db.

SQL – Structured Query Language

SQL – Structured Query Language, è il linguaggio che permette di effettuare le operazioni per estrarre e manipolare i dati da un database.

E' lo standard tra i sistemi relazionali: viene usato in tutti i prodotti DBMS come set di comandi per l'utente della base di dati

Tipi di istruzioni SQL

- ◉ **DCL** (Data control language): permette di gestire il controllo degli accessi e i permessi per gli utenti
- ◉ **DDL** (Data Definition Language): permette di definire la struttura del database
- ◉ **DML** (Data manipulation language): permette di modificare i dati contenuti nel db, con le operazioni di inserimento, variazione e cancellazione
- ◉ **TCL** (Transaction Control Language): queste operazioni gestiscono le transazioni nel database
- ◉ **Query Language**: permette di porre interrogazioni al db

DCL

gestire il controllo degli accessi e i permessi per gli utenti:

```
CREATE USER 'vecchione'@'host' IDENTIFIED BY 'password';
```

```
GRANT ALL  
ON nomedb.*  
TO 'user'@'host';
```

DDL

permette di definire la struttura del database

```
CREATE DATABASE dbName; --crea un nuovo database
DROP DATABASE dbName; --cancella il database
CREATE TABLE tableName(...); --crea una nuova tabella nel DB
ALTER TABLE tableName ... ; --modifica la struttura di una tabella
DROP TABLE tableName ... ; --cancella una tabella dal DB
CREATE INDEX indexName ... ; --crea un indice su una certa tabella
ALTER TABLE tableName DROP INDEX indexName ... ; --elimina l'indice specificato
```

DML

permette di *modificare i dati contenuti nel db*, con le operazioni di inserimento, variazione e cancellazione

- ◉ Inserimento:

```
INSERT INTO tableName(field1, field2, ...)  
VALUES ('value1', 'value2', '...');
```

- ◉ Cancellazione:

```
DELETE FROM tableName  
WHERE column_name = some_value;
```

- ◉ Aggiornamento:

```
UPDATE tableName  
SET column_name = new_value  
WHERE column_name = some_value;
```


TCL

gestiscono le transazioni nel database

```
COMMIT; --rende definitive le operazioni sul database  
ROLLBACK; --ripristina i dati eliminando le modifiche temporanee  
SAVEPOINT save_point_name(...); --crea un punto di salvataggio
```

Query Language

permette di porre interrogazioni al db

```
SELECT field(s)  
FROM table(s)  
WHERE condition(s);
```

Attraverso **SELECT** vengono selezionati dei campi (*attributi*) da una o più tabelle e restituiti all'utente sotto forma di una "nuova tabella" (*resultset*)

Attraverso la clausola **WHERE** è possibile filtrare il *resultset* sulla base di alcune regole

Installazione MySQL 8

Dal sito di [mysql](https://dev.mysql.com/downloads/mysql/) scaricate il pacchetto relativo al vostro sistema operativo (Linux, Windows o MacOS).

<https://dev.mysql.com/downloads/mysql/>

...

Il *client mysql* verrà installato nella cartella...



► C:\Program Files\MySQL\MySQL Server 8.0\bin



► /usr/local/mysql-8.0.26-macos10.15-x86_64/bin

dove si trova l'eseguibile di mysql

Avvio del servizio

MySQL viene eseguito come servizio o come *daemon*.

Un servizio o demone è un *programma in esecuzione continua* nel sistema operativo, il cui compito è quello di rimanere in attesa di richieste per la fruizione di specifiche funzionalità.

Il demone si chiama *mysqld*:

mysqld viene avviato in automatico all'avvio del sistema

(a seconda della configurazione applicata durante l'installazione)

Possiamo verificare che il servizio sia attivo con **mysqladmin**

```
mysqladmin -u root -p ping
```

Fornendo i corretti dati di autenticazione, se il DBMS è attivo, sarà stampato il messaggio "mysql is alive";

```
mysqld is alive
```

se invece non è attivo, verrà mostrato un errore di connessione.

```
mysqladmin: connect to server at 'localhost' failed  
error: 'Can't connect to local MySQL server through socket '/tmp/mysql.sock' (2)'  
Check that mysqld is running and that the socket: '/tmp/mysql.sock' exists!
```


Avvio/Arresto del servizio

MacOSX

Su mac è installato un controllo tra le Preferenze di sistema per gestire il servizio di MySQL.

Qui avviate/arrestate il servizio cliccando sul pulsante:

Stop / Start MySQL Server

Windows 10

Apri la finestra *Esegui/Start* usando il tasto: Win + R

Digita *services.msc*

Ora cerca il servizio MySQL in base alla versione installata.

Fare clic su *interrompi*, *avvia* o *riavvia* il servizio.



PATH MySQL

Il *client mysql* si trova nella cartella...

- ▶ C:\Program Files\MySQL\MySQL Server 8.0\bin

Ogni volta che di deve accedere al servizio, una volta aperto il terminale, bisogna portarsi nel punto in cui è stato installato mysql:

```
cd C:\Program Files\MySQL\MySQL Server 8.X\bin
```

A questo punto possiamo accedere al servizio avviando mysql.

Per evitare di digitare ogni volta il percorso bisogna creare una scorciatoia per la shell

In Windows bisogna *aggiungere una variabile di ambiente* relativa al path di MySQL.



PATH MySQL

Il *client mysql* si trova nella cartella...

- ▶ /usr/local/mysql-8.0.23-macos10.15-x86_64/bin

Ogni volta che di deve accedere al servizio, una volta aperto il terminale, bisogna portarsi nel punto in cui è stato installato mysql:

```
cd /usr/local/mysql-8.0.26-macos10.15-x86_64/bin/
```

A questo punto possiamo accedere al servizio avviando mysql (vedi slide 70)

Per creare una scorciatoia per la shell (impostare la shell bash) su mac bisogna creare un file nascosto ".bash_profile" con la scorciatoia scritta in questo modo:

```
export PATH=${PATH}:/usr/local/mysql/bin
```


Accesso al DBMS

Amministratore del servizio (DBA - Data Base Administrator)

Da terminale accedere a MySql:

```
mysql -u root -p
```

Vi verrà chiesto di inserire la password (è quella creata in fase di installazione)

```
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 16  
Server version: 8.0.23 MySQL Community Server - GPL  
  
Copyright (c) 2000, 2021, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql>
```

Visualizzate elenco database disponibili:

```
mysql> show databases;
```

Essendo una prima installazione dovrete vedere i seguenti db:

```
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
4 rows in set (0.01 sec)  
  
mysql>
```

I database elencati sono funzionali al servizio, sono disponibili solo all'amministratore del sistema;

information_schema e *performance_schema* sono disponibili anche all'utente per ottenere informazioni sul proprio database e sulle prestazioni delle query.

DDL

Data Definition Language (1parte)

Creare un database

Una volta effettuato l'accesso possiamo eseguire l'istruzione CREATE DATABASE (CREATE SCHEMA) seguita dal nome del database da creare.

```
CREATE DATABASE databaseName;
```

Se il database è già presente mysql ve lo segnala attraverso un messaggio di errore:

```
ERROR 1007 (HY000): Can't create database 'nomedatabase'; database exists
```

Usando la sintassi seguente

```
CREATE DATABASE IF NOT EXISTS databaseName;
```

mysql verifica l'esistenza del db: se non esiste lo crea, se esiste vi da ok ma segnala un *warning*.

```
Query OK, 1 row affected, 1 warning (0.01 sec)
```

In fase di creazione di un database con MySQL è anche possibile specificare charset e collation; ad esempio:

```
CREATE DATABASE IF NOT EXISTS databaseName  
CHARACTER SET utf8  
COLLATE utf8_general_ci;
```

Specificando questi valori è possibile "sovrascrivere" quelli impostati di default a livello server.

Per visualizzare come è stato creato il database

```
SHOW CREATE DATABASE databaseName;
```

Per elencare rispettivamente i set di caratteri disponibili e le "collezioni" disponibili.

```
SHOW CHARACTER SET; SHOW COLLATION;
```


Cancellare un database

Per eliminare un DB basta scrivere l'istruzione DROP DATABASE (DROP SCHEMA) seguita dal nome del database da rimuovere.

```
DROP DATABASE databaseName;  
DROP DATABASE IF EXISTS databaseName;
```

Se si usa l'istruzione opzionale **IF EXISTS** si evita di ricevere l'errore qualora il database sia già stato eliminato.

DCL

Data Control Language

DCL

gestire il controllo degli accessi e i permessi per gli utenti:

Istruzione **CREATE USER**: crea l'utente e assegna una password

```
CREATE USER 'app_java'@'localhost' IDENTIFIED BY 'java_2025!';
```

password : password associata all'utente che stiamo creando.

La password va scritta "in chiaro".

GRANT

Assegna i permessi e/o privilegi

```
GRANT --istruzioni_consentite  
ON namedatabase.*  
TO 'nameuser'@'hostuser';
```

istruzioni_consentite: CREATE, SELECT, UPDATE, DELETE, ALTER, DROP,...

Per dare all'utente permessi completi utilizzare la parola chiave ALL.

database.* : nome del database sul quale l'utente potrà eseguire le istruzioni consentite.

Per tutte le tabelle del db: .* . Si può specificare il nome di una o più tabelle. Per tutti i database: *.* .

user : specifica il nome dell'utente che vogliamo creare o al quale vogliamo assegnare nuovi permessi.

host : specifica il/gli host da cui è ammessa la connessione. Se voglio indicare più IP devo usare la wild card %: 130.192.200.%

REVOKE

```
REVOKE --istruzioni_revocate  
ON databaseName.*  
FROM 'user'@'host';
```

per la quale valgono le stesse regole sopra viste per **GRANT** .

Per eliminare tutti i privilegi:

```
REVOKE ALL PRIVILEGES, GRANT OPTION -- istruzioni_revocate  
FROM 'user'@'host';
```

Quest'ultima sintassi elimina ogni permesso dell'utente su qualunque database del sistema.

Cambiare/aggiornare la password MySQL degli utenti

Per cambiare una normale password utente devi digitare:

- ◉ *Cambia password per l'utente (da root):*

```
ALTER USER 'userName'@'host' IDENTIFIED BY 'newpass';
```

- ◉ Cambiare la propria password:

```
ALTER USER USER() IDENTIFIED BY 'newpass';
```


Verificare i permessi utente

Verificare i privilegi di uno specifico utente:

```
SHOW GRANTS FOR 'user'@'localhost';
```

Verificare i privilegi dell'utente attualmente loggato a MySQL:

```
SHOW GRANTS FOR CURRENT_USER;
```

Eliminare un utente da MySQL

```
DROP USER 'user'@'localhost';
```

questo comando rimuove l'utente e i suoi permessi.

Visualizzare elenco utenti mysql (solo utente root)

```
SELECT user, host FROM mysql.user;
```

Accesso al DBMS

Utente

Da terminale accedere a MySql utilizzando le credenziali dell'utente creato:

```
mysql -u nomeUser -p
```

Vi verrà chiesto di inserire la password (quella assegnata all'utente)

```
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 16  
Server version: 8.0.23 MySQL Community Server - GPL
```

```
Copyright (c) 2000, 2021, Oracle and/or its affiliates.
```

```
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```

```
mysql> show databases;
```


Accesso al DBMS - Utente

A questo punto dovete prendere possesso del database per poterci lavorare:

L'istruzione è **USE** più nome del database assegnato (senza **;** a chiusura del comando)

```
use nomeDatabase
```

Da terminale potete accedere *anche direttamente* al database per i quali avete i privilegi.

Dovete passare a mysql tutti i parametri (tranne la password) più il nome del database:

```
mysql -u nomeUser -p nomeDatabase
```

Non dovete più usare **USE**, siete nel database e potete lavorarci*

*le applicazioni, di solito, hanno un file dedicato alla connessione al db in cui vengono passati tutti i parametri di connessione compreso l'host e in alcuni casi anche la porta (dipende dal provider del servizio).

DDL

Data Definition Language (2 parte)

Creare le tabelle

Per creare una tabella usiamo il comando `CREATE TABLE tableName()`.

Quando creiamo una tabella dobbiamo definire anche tutti i campi ad essa associati, argomenti della parentesi.

- ◉ Per ogni campo verrà definito il dominio che indica al sistema quale tipo di dati verrà memorizzato nel campo.
- ◉ Per ogni campo definiamo anche gli eventuali attributi

```
CREATE TABLE IF NOT EXISTS nome_tabella(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  fieldName1 VARCHAR(60) NOT NULL,  
  fieldName2 DATE,  
  fieldName3 TINYINT  
)  
[ENGINE InnoDB CHARACTER SET utf8 COLLATE utf8_general_ci];
```


Esempio di creazione di una tabella denominata studente, con il campo id come chiave primaria.

```
CREATE TABLE IF NOT EXISTS studenti(  
    id INT AUTO_INCREMENT,  
    nome VARCHAR(20),  
    cognome VARCHAR(30) NOT NULL,  
    genere ENUM('m', 'f'),  
    indirizzo VARCHAR(100),  
    citta VARCHAR(30),  
    provincia CHAR(2) DEFAULT 'To',  
    regione VARCHAR(30) DEFAULT 'Piemonte',  
    email VARCHAR(100) NOT NULL UNIQUE,  
    data_nascita date,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    PRIMARY KEY(id)  
);
```

Verificare se e come è stata creata la tabella:

- ◉ Per verificare se la nostra tabella è stata creata si usa l'istruzione che mostra le tabelle presenti:

```
SHOW TABLES;
```

- ◉ Per verificare che la struttura della tabella sia corretta:

```
SHOW CREATE TABLE tableName;
```

- ◉ Per visualizzare come è stata creata una tabella:

```
DESCRIBE tableName; DESC tableName;
```

Rinominare una tabella:

```
ALTER TABLE tableName RENAME newtableName;
```

```
RENAME TABLE tableName TO newtableName;
```

INFORMAZIONI SULLE TABELLE

Per conoscere la struttura della tabella con più o meno informazioni (valore dell'auto_increment, data di creazione, collation)

```
DESCRIBE tableName; DESC tableName;
```

```
SHOW COLUMNS FROM tableName;
```

→ *Mostrano le stesse informazioni*

```
SHOW FULL COLUMNS FROM tableName;
```

→ *Mostra ulteriori informazioni (privilegi e commenti)*

```
SHOW INDEX FROM tableName;
```

```
SHOW TABLE STATUS LIKE 'tableName'1; -- mostra valore auto_increment
```

Per conoscere solo il valore dell'auto_increment di una tabella possiamo interrogare anche il db INFORMATION_SCHEMA

```
SELECT table_name, auto_increment  
FROM information_schema.tables  
WHERE table_schema = 'databaseName'1;  
-- AND TABLE_NAME = 'tableName';
```

1) Le statistiche della tabella vengono memorizzate nella cache. Per disabilitare la cache e avere sempre l'ultima versione è necessario modificare la variabile del server che indica la durata del cache-clear a 0:

```
SET PERSIST information_schema_stats_expiry = 0 -- bisogna usare l'utente DBA (root)
```


INFORMATION_SCHEMA

INFORMATION_SCHEMA fornisce l'accesso ai metadati del database, le informazioni sul server MySQL come il nome di un database o una tabella, il tipo di dati di una colonna, o privilegi di accesso...

Note sull'utilizzo INFORMATION_SCHEMA

- ◉ INFORMATION_SCHEMA è una banca dati all'interno di ciascuna istanza di MySQL, il **luogo che memorizza le informazioni su tutti gli altri database che il server MySQL mantiene**.
Il database INFORMATION_SCHEMA contiene diverse tabelle di sola lettura (in realtà viste).
- ◉ Anche se è possibile selezionare INFORMATION_SCHEMA come database predefinito con un'istruzione USE, *è possibile SOLO leggere il contenuto delle tabelle*, non eseguire INSERT, UPDATE o DELETE.

INFORMATION_SCHEMA

Vediamo un esempio di utilizzo di INFORMATION_SCHEMA

```
SELECT table_name, table_type, engine, table_collation  
FROM information_schema.tables  
WHERE table_schema = 'databaseName'  
ORDER BY table_name;
```

Questa select ci restituirà l'elenco delle tabelle del nostro database indicando il tipo di tabella e l'engine associato.

Modificare le tabelle

L'istruzione **ALTER TABLE** viene utilizzata per aggiungere, eliminare o modificare le colonne di una tabella esistente.

Per *aggiungere un campo a una tabella*, utilizzare la seguente sintassi:

```
ALTER TABLE tableName  
ADD fieldName [DATATYPE];
```

Per modificare *nome e datatype* di un campo di una tabella

```
ALTER TABLE tableName  
CHANGE fieldName newFieldName [DATATYPE];
```

Per modificare *solo datatype* di un campo di una tabella

```
ALTER TABLE tableName  
MODIFY fieldName [DATATYPE];
```

Per modificare *solo nome* di un campo di una tabella

```
ALTER TABLE tableName  
RENAME COLUMN oldName TO newName;
```

Per cancellare un campo di una tabella

```
ALTER TABLE tableName  
DROP fieldName;
```

Potete combinare le istruzioni separandole con una virgola

```
ALTER TABLE tableName  
CHANGE fieldName newFieldName [DATATYPE],  
ADD fieldName3 [DATATYPE] AFTER fieldName2,  
DROP fieldName4;
```

Per aggiungere un campo in una data posizione

```
ALTER TABLE tableName  
ADD fieldName2 [DATATYPE]  
AFTER fieldName;
```

Usando *FIRST* al posto di *AFTER* si aggiunge il campo in prima posizione, *FIRST* non vuole il nome del campo

Per spostare un campo in una tabella

```
ALTER TABLE tableName  
MODIFY fieldName2 [DATATYPE]  
AFTER fieldName;
```

Per spostare un campo in prima posizione

```
ALTER TABLE tableName  
MODIFY fieldName2 [DATATYPE]  
FIRST;
```

se il campo è una chiave primaria non dovete indicarlo nel datatype altrimenti ricevete un errore di mysql

- Aggiungere alla tabella la PRIMARY KEY (se non già impostata):

```
ALTER TABLE tableName ADD PRIMARY KEY (field1[, field2, ...]);
```

- Aggiungere alla tabella la PRIMARY KEY aggiungendo campo nuovo apposito:

```
ALTER TABLE tableName  
ADD id INT AUTO_INCREMENT PRIMARY KEY;
```

- Eliminare la PRIMARY KEY

```
ALTER TABLE tableName DROP PRIMARY KEY;
```

Duplicare tabelle

Se abbiamo necessità di duplicare una tabella possiamo utilizzare l'istruzione **CREATE TABLE** combinata con l'istruzione **LIKE**.

Per duplicare una tabella possiamo scrivere:

```
CREATE TABLE tableNameCopy LIKE tableName;
```

Questa istruzione duplica solo la struttura della tabella.

Cancellare una tabella dal database MySQL

Vediamo l'operazione inversa alla creazione di tabelle, la loro eliminazione.

- Per eliminare una tabella utilizzeremo il comando DROP TABLE.

```
DROP TABLE tableName;
```

L'eliminazione di una tabella, come per il database, è un'operazione irreversibile

- È possibile eliminare più di una tabella contemporaneamente:

```
DROP TABLE tableName, tableName2, tableName3;
```


DML

Data Manipulation Language

Creazione, lettura, aggiornamento e eliminazione dei record (CRUD)

Una volta creata la struttura del nostro database ci ritroveremo, ovviamente, con una serie di tabelle vuote.

Prima di aggiungere record a una tabella bisogna conoscere il tipo di dati previsto per ogni campo, quali campi non possono avere valore nullo, quali campi hanno l'incremento automatico...

Quando si inseriscono i dati bisogna usare le *virgolette* o gli *apici* per i dati *tipo stringa* (compresa la data), *senza virgolette* o *apici* per i dati di *tipo numerico*.

Non si inseriscono i valori per i campi definiti con l' *auto_increment*.

Consistenza dei dati

La consistenza dei dati è una proprietà fondamentale nei database che garantisce che i dati rispettino regole e vincoli predefiniti, mantenendo uno stato valido e coerente.

In altre parole, **i dati devono sempre essere corretti e rispettare le regole definite dal database.**

```
CREATE TABLE studenti (  
    id INT PRIMARY KEY,  
    nome VARCHAR(50),  
    età TINYINT UNSIGNED CHECK (età >= 18) -- Vincolo di consistenza  
);
```

La consistenza fa parte delle proprietà ACID (Atomicità, Consistenza, Isolamento, Durabilità), fondamentali per garantire la corretta esecuzione delle transazioni nei database relazionali.

INSERT INTO

INSERT INTO è l'istruzione utilizzata per inserire nuovi record in una tabella. Ha due parti

INSERT INTO seleziona la tabella e i campi per i quali effettuare l'inserimento

VALUE/VALUES elenca i valori dei campi da inserire

```
INSERT INTO tableName(field1, field3)
VALUES(value1, value3);
```

È possibile inserire più record con un solo INSERT separando l'elenco dei valori di ogni record con la: ,

```
INSERT INTO tableName(field1, field2, field3,...)
VALUES(r1_value1, r1_value2, r1_value3, ...),(r2_value1, r2_value2, r2_value3, ...);
```

Altra sintassi per singolo record con istruzione **SET**:

```
INSERT INTO tableName
SET field1 = 'value1', field2 = 'value2', field3 = 'value2';
```

INSERT INTO

È possibile usare il comando **INSERT INTO** senza l'uso di nomi di campo se si inserisce un record rispettando l'ordine dei campi della tabella

```
INSERT INTO tableName  
VALUES(value1, value2, value3);
```

in questo caso devono essere inseriti i valori di tutti i campi, anche i valori *AUTO_INCREMENT* o *TIMESTAMP* (passare default per inserimento automatico).

Per i campi che accettano i valori nulli potete passare `null`.

```
INSERT INTO studente  
VALUES(default, 'fabio', 'rossi', 'fbr@gmail.com', null, default);
```


Mostrare i record di una tabella

È possibile visualizzare i record di una tabella utilizzando l'istruzione **SELECT**.

Per visualizzare tutti i record da una tabella si usa il carattere jolly *.

Dobbiamo anche utilizzare l'istruzione **FROM** per identificare la tabella che vogliamo interrogare:

```
SELECT * FROM tableName;
```

Di solito si visualizzano campi specifici, piuttosto che l'intera tabella.

Dopo l'istruzione SELECT elencare i campi che interessano, separati da una virgola.

```
SELECT fieldName, fieldName2, fieldName3 FROM tableName;
```

INSERT INTO ... SELECT

Inserire i dati prendendoli da un'altra tabella:

```
INSERT INTO amici(nome, cognome)  
SELECT nome, cognome  
FROM studenti;
```

Nell'esempio qui sopra abbiamo immaginato di popolare di dati la tabella *amici* inserendo automaticamente i dati già presenti nella tabella *studenti*.

È necessario che i campi nelle due tabelle contengano lo stesso tipo di dato e che la tabella *amici* esista.

CREATE TABLE ... SELECT

Si possono creare delle tabelle già popolate di dati mediante l'uso congiunto delle istruzioni **CREATE TABLE** e **SELECT**.

CREATE TABLE crea la nuova tabella,
SELECT carica i dati prelevandoli da un'altra tabella.

Il suo funzionamento, in pratica, è analogo a quello di **INSERT INTO ... SELECT**.

```
CREATE TABLE parenti (  
    id INT AUTO_INCREMENT,  
    nome VARCHAR(20),  
    cognome VARCHAR(30),  
    PRIMARY KEY(id)  
) SELECT nome, cognome FROM amici;
```


Duplicare tabelle e suoi contenuti

Se abbiamo necessità di copiare il contenuto di una tabella in altra tabella, possiamo utilizzare l'istruzione **CREATE TABLE** combinata con **LIKE** e le istruzioni **SELECT**.

Per duplicare esattamente una tabella(con indici e chiavi) e i suoi contenuti bisogna usare due istruzioni separate:

```
CREATE TABLE studenti_bk LIKE studenti;  
INSERT INTO studenti_bk SELECT * FROM studenti;
```

Si può usare anche un'istruzione sola: in questo caso gli indici non vengono ricreati, cioè le strutture delle tabelle sono diverse:

```
CREATE TABLE studenti_bk2 [AS]  
SELECT * FROM studenti;
```

UPDATE

Aggiornamento dei record in una tabella.

Questa istruzione modifica il valore presente in una colonna di un record già esistente.

Viene utilizzata insieme all'istruzione **SET**:

```
UPDATE tableName  
SET field1 = value1, field2 = value2  
WHERE field3 = value3;
```

- dopo **UPDATE** indichiamo quale tabella è interessata
- con **SET** specifichiamo quali colonne modificare e quali valori assegnare
- con **WHERE**(opzionale) stabiliamo le condizioni che determinano quali righe saranno interessate dalle modifiche (att: **se non specifichiamo una condizione tutte le righe saranno modificate**)

Per operare simultaneamente su più campi è sufficiente suddividere le coppie chiave/valore con una virgola.

Quando si inseriscono i dati in una tabella rammentate sempre come sono stati definiti gli attributi per evitare errori di inserimento.

Se si inserisce un valore *troppo lungo*, o *non compreso* dalla definizione dell'attributo, MySQL restituisce un errore* e non effettua alcuna modifica.

```
UPDATE studenti SET genere = 's' WHERE id = 1;  
ERROR 1265 (01000): Data truncated for column 'genere' at row 1
```

Il campo *genere* della tabella *studenti* è un campo definito come:

`ENUM( 'm', 'f' )` , accetta quindi solo i valori *m* o *f*.

In questo caso stiamo tentando di inserire un valore non ammesso.

*dipende dall'impostazione della variabile globale @@sql_mode: di default mysql lavora in strict mode.

SQL Mode: STRICT MODE

Il server MySQL può funzionare in diverse modalità SQL e può applicare queste modalità in modo diverso per client diversi, a seconda del valore della variabile di sistema: `SQL_MODE`.

I DBA possono impostare la modalità SQL globale in modo che corrisponda ai requisiti operativi del server del sito e ogni applicazione può impostare la modalità SQL della sessione in base ai propri requisiti.

Le modalità influiscono sulla sintassi SQL supportata da MySQL e dai controlli di convalida dei dati che esegue. Ciò semplifica l'utilizzo di MySQL in ambienti diversi e l'utilizzo di MySQL insieme ad altri server di database.

```
SELECT @@SQL_MODE;
+-----+
| @@SQL_MODE |
+-----+
| NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTION |
+-----+
```

Questo significa che se volete operare con un sessione che lavori in STRICT MODE (che attivi i controlli sui campi per esempio) dovete impostare la variabile ad inizio sessione di connessione:

```
SET SQL_MODE = 'TRADITIONAL';
```

<https://dev.mysql.com/doc/refman/8.0/en/sql-mode.html>

Eliminazione dei record in una tabella

L'istruzione **DELETE** viene utilizzata per eliminare gruppi di record in una tabella.

È necessario utilizzare la parola chiave condizionale **WHERE** per isolare quali record si desidera eliminare, altrimenti si eliminano tutti i record.

La sintassi di base per l'istruzione è la seguente :

```
DELETE  
FROM tableName  
WHERE field = value;
```

esempio:

```
DELETE  
FROM studenti  
WHERE genere = 'm';
```


Eliminare tutti i record della tabella

Per svuotare una tabella si usa l'istruzione **TRUNCATE**

```
TRUNCATE [TABLE] tableName;
```

Questa soluzione è la più veloce perché elimina la struttura della tabella per poi ricrearne una uguale vuota azzerando il valore di eventuali campi AUTO_INCREMENT.

Usando **DELETE** si eliminano tutti i record presenti nella tabella specificata *record per record*.

```
DELETE FROM tableName;
```

Un simile modo di operare, seppur assolutamente funzionante, è poco efficiente perché dipende dalla quantità di righe presenti in tabella.

Inoltre usando DELETE il valore di un eventuale AUTO_INCREMENT rimane inalterato; per azzerarlo:

```
ALTER TABLE tableName AUTO_INCREMENT = 1;
```

INFORMAZIONI SULLE TABELLE

Per conoscere la struttura della tabella con più o meno informazioni (valore dell'auto_increment, data di creazione, collation)

```
DESCRIBE tableName; DESC tableName;
```

```
SHOW COLUMNS FROM tableName;
```

```
SHOW FULL COLUMNS FROM tableName;
```

```
SHOW INDEX FROM tableName;
```

```
SHOW TABLE STATUS LIKE 'tableName'1; -- mostra valore auto_increment
```

Per conoscere solo il valore dell'auto_increment di una tabella possiamo interrogare anche il db INFORMATION_SCHEMA

```
SELECT table_name, auto_increment  
FROM information_schema.tables  
WHERE table_schema = 'databaseName'1;  
-- AND TABLE_NAME = 'tableName';
```

1) Le statistiche della tabella vengono memorizzate nella cache. Per disabilitare la cache e avere sempre l'ultima versione è necessario modificare la variabile del server che indica la durata del cache-clear a 0:

```
SET PERSIST information_schema_stats_expiry = 0 -- bisogna usare l'utente DBA (root)
```

Commenti

MySQL Server supporta tre stili di commento:

- `#` da questo simbolo a fine riga.
- `--` da questo simbolo a fine riga.
lo stile di commento (doppio trattino) richiede che il secondo trattino sia seguito da almeno uno spazio bianco o un carattere di controllo (come uno spazio, tab, nuova riga e così via).
- `/* commento */` come nel linguaggio C.
Questa sintassi consente un commento su più righe e l'inserimento del commento *inline*.

L'esempio seguente mostra tutti e tre gli stili di commento:

```
SELECT 1 + 1; # Questo commento continua fino alla fine della riga
SELECT 1 + 1; -- Questo commento continua fino alla fine della riga
SELECT 1 /* questo è un commento in linea */ + 1;
SELECT 1 +
/*
questo è un commento
su più linee
*/
1;
```


Importare dati da file

Sintassi per importare dati presenti in altri file, come file di testo o CSV (Comma Separate Value)¹.

```
LOAD DATA
  [LOW_PRIORITY | CONCURRENT] [LOCAL]
  INFILE 'file_name'
  INTO TABLE tbl_name
  [{FIELDS | COLUMNS}
    [TERMINATED BY 'string']
    [[OPTIONALLY] ENCLOSED BY 'char']
    [ESCAPED BY 'char']
  ]
  [LINES
    [STARTING BY 'string']
    [TERMINATED BY 'string']
  ]
  [IGNORE number {LINES | ROWS}]
```

Potreste non avere i privilegi necessari per usare questo comando²{slide successiva}.

Devono essere rispettate alcune condizioni:

- la tabella in cui importare i dati deve già esistere;
- gli attributi devono essere dello stesso tipo;

¹) approfondimento con altre opzioni: <https://dev.mysql.com/doc/refman/8.0/en/load-data.html>

² verificare il valore della variabile che consente l'operazione:

```
SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

se 'OFF' impostare il valore ad ON.

```
SET GLOBAL local_infile = 'ON';
```

Per il client MySQLWorkbench occorre configurare la connessione al server e aggiungere la riga seguente

```
OPT_LOCAL_INFILE=1
```

al box "Others" del tab "advanced"

Da terminale bisogna aggiungere i parametri `--local-infile` quando ci si connette al servizio:

```
mysql --local-infile -u [user] -p
```


Vediamo un esempio:

Creiamo una tabella `studente_imp`; importiamo i dati dal file *data.txt*

```
LOAD DATA LOCAL  
INFILE 'c:/Users/anskat_PC/Desktop/data.txt'  
INTO TABLE studenti_imp  
FIELDS TERMINATED BY ','  
OPTIONALLY ENCLOSED BY ''''  
LINES TERMINATED BY '\n';
```

Il file *data.txt* deve contenere tutti i valori per le colonne presenti nella tabella `studente_imp`

(usare l'opzione *LOCAL* per caricare il file dal client)

Query

Query Language - interrogazione dei dati

Interrogazione dei dati

Creazione di query di base

Abbiamo già introdotto il comando SELECT per visualizzare i record inseriti in una tabella.

```
SELECT field1, field2, field3 FROM tableName;
```

Utilizzando il carattere jolly * selezioniamo tutte le colonne dei campi di dati da visualizzare.

```
SELECT * FROM tableName;
```


ORDER BY

Consente di ordinare i risultati di una query.

L'istruzione *ORDER BY* è seguita dal(dai) campo(i) su cui si basa l'ordinamento.

L'ordine predefinito è crescente (ASC).

```
SELECT *  
FROM studenti  
ORDER BY cognome;
```

Usando l'istruzione **DESC** si ordina in modo decrescente:

```
SELECT *  
FROM studenti  
ORDER BY cognome DESC;
```

Se si vuole ordinare per nome con ordinamento *DESC* e per eta con ordinamento *ASC* dobbiamo scrivere:

```
SELECT *  
FROM studenti  
ORDER BY cognome DESC, eta;
```

LIMIT

Consente di definire il numero dei record massimo da visualizzare.

Accetta due argomenti: [i] , opzionale, specifica l'indice da cui partire per mostrare i record.

Il secondo argomento indica la quantità di record da mostrare:

```
SELECT *  
FROM tableName  
ORDER BY field  
LIMIT [i],10;
```

La query seguente mostra solo i primi 10 record della tabella *studente*;

```
SELECT *  
FROM studenti  
ORDER BY cognome  
LIMIT 10;
```

La query seguente mostra 10 record della tabella *studente* presi a partire dall'*undicesimo record*.

Ricordate che l'indice parte da 0, per cui l'undicesimo record è l'indice numero 10 .

```
SELECT *  
FROM studenti  
ORDER BY cognome  
LIMIT 10 , 10;
```

SELECT e WHERE

WHERE consente di filtrare i risultati di una query, *mostrando solo i record che soddisfano la condizione imposta.*

Se vogliamo selezionare il nome e cognome degli studenti solo di genere maschile possiamo applicare un filtro sull'attributo genere come condizione del **WHERE** grazie all'uso degli operatori.

```
SELECT nome, cognome  
FROM studenti  
WHERE genere = 'm';
```


Operatori

- operatori di confronto

`=, <>, !=, >, >=, <, <=`

- operatori logici

`AND, OR, NOT`

- operatori di confronto avanzato

`IN, NOT IN, BETWEEN, NOT BETWEEN, IS NULL, IS NOT NULL,
LIKE, NOT LIKE, REGEXP`

- operatori matematici

`+, -, *, /, %`

Operatori di confronto

Operatore	Descrizione
=	Equal
<>	Not Equal
!=	Not Equal
>	Greater Than
>=	Greater Than or Equal
<	Less Than
<=	Less Than or Equal

operatori confronto

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE genere = 'f'  
ORDER BY cognome, nome;
```

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE citta <> 'torino'  
ORDER BY cognome, nome;
```

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE data_nascita >= '1989-12-31'  
ORDER BY data_nascita;
```

operatori logici: **AND** e **OR**

Quando si utilizza l'operatore **AND** **tutte le condizioni** devono essere vere.

```
SELECT field(s)
FROM tableName
WHERE condition1 AND condition2 AND condition3;
```

```
SELECT nome, cognome, email, data_nascita
FROM studenti
WHERE genere = 'm' AND provincia = 'to';
```

Vengono filtrati i record degli studenti maschi provenienti dalla provincia di Torino.

Quando si utilizza l'operatore **OR** **almeno una delle condizioni** deve essere vera.

```
SELECT nome, cognome, email, data_nascita
FROM studenti
WHERE genere = 'm' OR provincia = 'to';
```

Vengono filtrati i record degli studenti maschi e degli studenti che provengono dalla provincia di Torino (tra cui potrebbero esserci delle studentesse).

Attenzione: l'operatore AND ha precedenza su OR, quindi è consigliabile usare le parentesi per una migliore leggibilità.

Esempio senza parentesi (potenzialmente ambiguo):

```
SELECT * FROM studenti  
WHERE cognome = 'verdi' OR cognome = 'rossi'  
AND provincia = 'to'; -- AND ha precedenza
```

Meglio con le parentesi per chiarire l'ordine della ricerca:

```
SELECT * FROM studenti  
WHERE cognome = 'verdi' OR (cognome = 'rossi'  
AND provincia = 'to'); -- AND ha precedenza
```

Esempio con più condizioni:

```
SELECT * FROM studenti  
WHERE (cognome = 'verdi' OR cognome = 'rossi')  
AND (provincia = 'to' OR provincia = 'al');
```


operatori logici: **NOT**

L'operatore **NOT** **nega** le condizioni poste.

Questa query seleziona alcuni attributi degli studenti il cui genere **non** è 'm'

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE NOT genere = 'm';
```

Questa query seleziona alcuni attributi degli studenti il cui genere non è 'm' e la cui provincia è 'to'

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE NOT genere = 'm' AND provincia = 'to';
```

Questa query seleziona alcuni attributi degli studenti il cui genere **non** è 'm' e la cui provincia **non** è 'to'

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE NOT genere = 'm' AND NOT provincia = 'to';
```

Se volete negare una o più condizioni è preferibile usare l'operatore <> o !=, il NOT può confondere e rendere meno leggibile la query

Operatori di confronto avanzati: **IN**, **NOT IN**

L'operatore **IN** ci consente di selezionare i record indicando più valori di campo.

Possiamo farlo con l'operatore OR, ma può diventare complicato quando confrontiamo molti valori.

Prendiamo ad esempio l'esecuzione di una query in cui cerchiamo solo gli studenti di alcune province:

```
SELECT nome, cognome, email, data_nascita, provincia
FROM studenti
WHERE provincia IN ('to', 'cn', 'at', 'no');
```

L'operatore **NOT IN** funziona in modo simile a **IN**, mostra i record che NON contengono i valori selezionati.

Così la seguente query mostrerà tutti i record di studenti che *non appartengono alle province in elenco*.

```
SELECT nome, cognome, email, data_nascita, provincia
FROM studenti
WHERE provincia NOT IN ('to', 'cn', 'at', 'no');
```

ATTENZIONE: *NOT IN* non considera i valori *NULL*, se si cercano anche quelli bisogna aggiungere ulteriore condizione.

BETWEEN, NOT BETWEEN

Utilizzando l'operatore BETWEEN possiamo selezionare un intervallo di valori. I valori di inizio e fine dell'intervallo sono inclusi. I valori dell'intervallo possono essere numeri, testo o date.

ATTENZIONE: *NOT BETWEEN* non considera i valori *NULL*.

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE data_nascita BETWEEN '1985-01-01' AND '1994-12-31';
```

IS NULL e IS NOT NULL

L'operatore **IS NULL** viene utilizzato per visualizzare i record che non hanno un valore impostato per un campo.

Viceversa **IS NOT NULL** mostra i record che hanno un valore impostato per un campo.

```
SELECT nome, cognome, email, data_nascita  
FROM studenti  
WHERE data_nascita IS NULL;
```

Questi operatori possono essere utilizzati per trovare i record che hanno bisogno di informazioni aggiuntive.

LIKE, NOT LIKE

L'operatore LIKE permette di effettuare *ricerche testuali con pattern matching*.

ATTENZIONE: *NOT LIKE* non considera i valori *NULL*.

```
SELECT * FROM studenti WHERE cognome LIKE 'v%';
SELECT * FROM studenti WHERE nome LIKE '%a';
SELECT * FROM studenti WHERE indirizzo LIKE 'via %';
SELECT * FROM studenti WHERE email LIKE '%gmail.com';
SELECT * FROM studenti WHERE nome LIKE 'paol_';
SELECT * FROM studenti WHERE nome LIKE '_a%';
```

La differenza è data dalla posizione del carattere percentuale (%) che sta ad indicare "qualsiasi carattere dopo" e "qualsiasi carattere prima".
L'uso di underscore (_) indica un solo carattere.

- non abusarne.
- i criteri di ricerca che iniziano con caratteri jolly sono quelli con i tempi di elaborazione più lunghi.

Wildcard	Descrizione
%	sostituisce zero o più caratteri
_	sostituisce un singolo carattere
[elenco caratteri]	pattern di caratteri da trovare (con wildcard)

REGEXP_LIKE() (REGEXP e RLIKE sono sinonimi di REGEXP_LIKE()).

Un operatore più potente di LIKE è **REGEXP** il quale consente di utilizzare molti più simboli per ricerche più complesse.

```
SELECT * FROM studenti WHERE REGEXP LIKE(nome,'ra'); SELECT * FROM studenti WHERE nome REGEXP 'ra';  
  
SELECT * FROM studenti WHERE nome REGEXP '^mar';  
  
SELECT * FROM studenti WHERE nome REGEXP 'co$';  
  
SELECT * FROM studenti WHERE nome REGEXP 'mar|ara|ola';
```

Potete combinare i simboli

```
SELECT * FROM studenti WHERE nome REGEXP '^mar|ara|co$';  
  
SELECT * FROM studenti WHERE nome REGEXP '^(mar|ara)|co$'; -- comincia per 'mar' o 'ara' o finisce con 'co'
```

Creare combinazioni di pattern

```
SELECT * FROM studenti WHERE nome REGEXP '[mcp]a';  
  
SELECT * FROM studenti WHERE nome REGEXP 'a[ero]';  
  
SELECT * FROM studenti WHERE nome REGEXP 'l[ao]$';  
  
SELECT * FROM studenti WHERE nome REGEXP '^[a-m]'; -- '^[n-z]'
```

Approfondimento:

https://dev.mysql.com/doc/refman/8.0/en/regexp.html#operator_regexp
<https://dev.mysql.com/doc/refman/8.0/en/regexp.html#regexp-syntax>

Operatori matematici

MySQL supporta i classici operatori matematici tradizionali, cioè:

- + (addizione)
- (sottrazione)
- * (moltiplicazione)
- / (divisione)
- % (modulo - il resto della divisione tra due numeri)

Questi operatori risultano molto utili quando, ad esempio, si devono svolgere dei calcoli all'interno di una SELECT. Per fare un esempio si supponga di voler restituire il valore dato dalla sottrazione di due campi:

```
SELECT (field1 - field2)
FROM tableName
[WHERE condition(s)];
```

Potete eseguire dei calcoli matematici con SELECT:

```
SELECT 6 / 2 ; ##3
```

```
SELECT 35 % 3 ; ##2
```

```
SELECT (35 / 3) * 2 ; ##11.6667
```

Calcolo campi al volo

Utilizzando il comando `SELECT` possiamo aggiungere campi alla nostra query che rappresentano calcoli algebrici di base su un campo esistente.

Nell'esempio che segue la query restituirà il prezzo corrente e il prezzo con il 10% in più.

```
SELECT titolo, prezzo, prezzo * 1.1  
FROM libro;
```

Usare gli operatori permette di fare un calcolo per ogni campo

Colonne generate (virtuali e persistenti / memorizzate)

Una **colonna generata** è una colonna in una tabella che non può essere impostata esplicitamente su un valore specifico in una query DML.

Il suo valore *viene invece generato automaticamente in base a un'espressione*. Questa espressione potrebbe generare il valore in base ai valori di altre colonne nella tabella oppure potrebbe generare il valore chiamando funzioni incorporate o funzioni definite dall'utente (UDF) .

Esistono due tipi di colonne generate:

- **PERSISTENT** (STORED): il valore di questo tipo è effettivamente memorizzato nella tabella.
- **VIRTUAL**: Il valore di questo tipo non viene memorizzato affatto. Il valore viene invece generato dinamicamente quando viene eseguita una query sulla tabella. Questo tipo è l'impostazione predefinita.

Le colonne generate sono talvolta chiamate anche colonne calcolate o colonne virtuali.

Sintassi

```
column_name data_type [GENERATED ALWAYS] AS ( <expression> )  
[VIRTUAL | STORED] [NOT NULL | NULL] [UNIQUE [KEY]] [PRIMARY KEY] [COMMENT <text>]
```

Esempio sulla tabella *studente*. Creiamo un campo generato al volo che contenga il nome e il cognome dello studente:

```
ALTER TABLE studente ADD fullName VARCHAR(255) AS (CONCAT(nome, ' ', cognome))  
AFTER cognome;
```

Esempio sulla tabella *libro*. Creiamo un campo generato al volo che calcoli il prezzo del libro compreso di IVA:

```
ALTER TABLE libro  
ADD prezzo_iva DECIMAL(6,2) GENERATED ALWAYS AS (prezzo * 1.10) STORED  
AFTER prezzo;
```

approfondimenti: <https://dev.mysql.com/doc/refman/8.0/en/create-table-generated-columns.html>

Limiti nella scrittura dell'espressione

- ◉ Sono consentiti valori letterali, funzioni integrate deterministiche e operatori. Una funzione è deterministica se, dati gli stessi dati nelle tabelle, più invocazioni producono lo stesso risultato, indipendentemente dall'utente connesso.
Esempi di funzioni che non sono deterministiche e non soddisfano questa definizione:
`CONNECTION_ID()`, `CURRENT_USER()`, `NOW()`, `CURDATE()`...
- ◉ Non sono consentite subquery.
- ◉ Una definizione di colonna generata può fare riferimento ad altre colonne generate, ma solo a quelle presenti in precedenza nella definizione di tabella.
Una definizione di colonna generata può fare riferimento a qualsiasi colonna di base (non generata) nella tabella indipendentemente dal fatto che la sua definizione sia precedente o successiva.
- ◉ L'attributo `AUTO_INCREMENT` non può essere utilizzato in una definizione di colonna generata.
- ◉ Non è possibile utilizzare una colonna `AUTO_INCREMENT` come colonna di base in una definizione di colonna generata.
- ◉ Se la valutazione dell'espressione provoca il troncamento o fornisce un input errato a una funzione, l'istruzione `CREATE TABLE` termina con un errore e l'operazione DDL viene rifiutata.

Limiti nella scrittura dell'espressione

La maggior parte delle espressioni deterministiche legali che possono essere calcolate sono supportate nelle espressioni per le colonne generate.

La maggior parte delle funzioni integrate è supportata nelle espressioni per le colonne generate.

Tuttavia, alcune funzioni integrate non possono essere supportate per motivi tecnici. Ad esempio, se si tenta di utilizzare una funzione non supportata in un'espressione, viene generato un errore simile al seguente:

```
ERROR 1901 (HY000): Function or expression 'dayname()' cannot be used in the GENERATED ALWAYS AS clause of `v`
```

Vediamo un esempio:

```
ALTER TABLE studenti ADD eta TINYINT AS (TIMESTAMPDIFF(YEAR,data_nascita,CURDATE()))  
VIRTUAL;
```

Non si può usare l'espressione, perché utilizza la funzione **CURDATE()** non deterministica

Interrogare più tabelle

STUDENTI

● id	cognome	nome	email
1	Esposito	Giuseppe	esposito.g@icloud.com
2	Pautasso	Carlo	cpautasso@gmail.com
3	Piras	Efisio	efpiras@gmail.com
4	Conti	Renzo	conti.renzo@icloud.com
5	Rigon	Paolo	prigon@gmail.com

ISCRIZIONI

corso_id	studente_id	prezzo	data
1	1	180.00	22/9/2022
1	2	180.00	23/9/2022
3	1	300.00	25/9/2022
2	3	120.00	28/9/2022

DOCENTI

● id	cognome	nome	email
1	Rossi	Mario	rossi.mario@icloud.com
2	Verdi	Paola	verdi.paola@gmail.com
3	Bruni	Marco	bruni.marco@gmail.com
4	Bianchi	Daniele	bianchi.daniele@icloud.com

CORSI

● id	titolo	prezzo	docente_id
1	Php	200.00	4
2	Javascript	120.00	2
3	Java	300.00	3

Legame logico tra tabelle

CORSI

Chiave Esterna

Campo Chiave

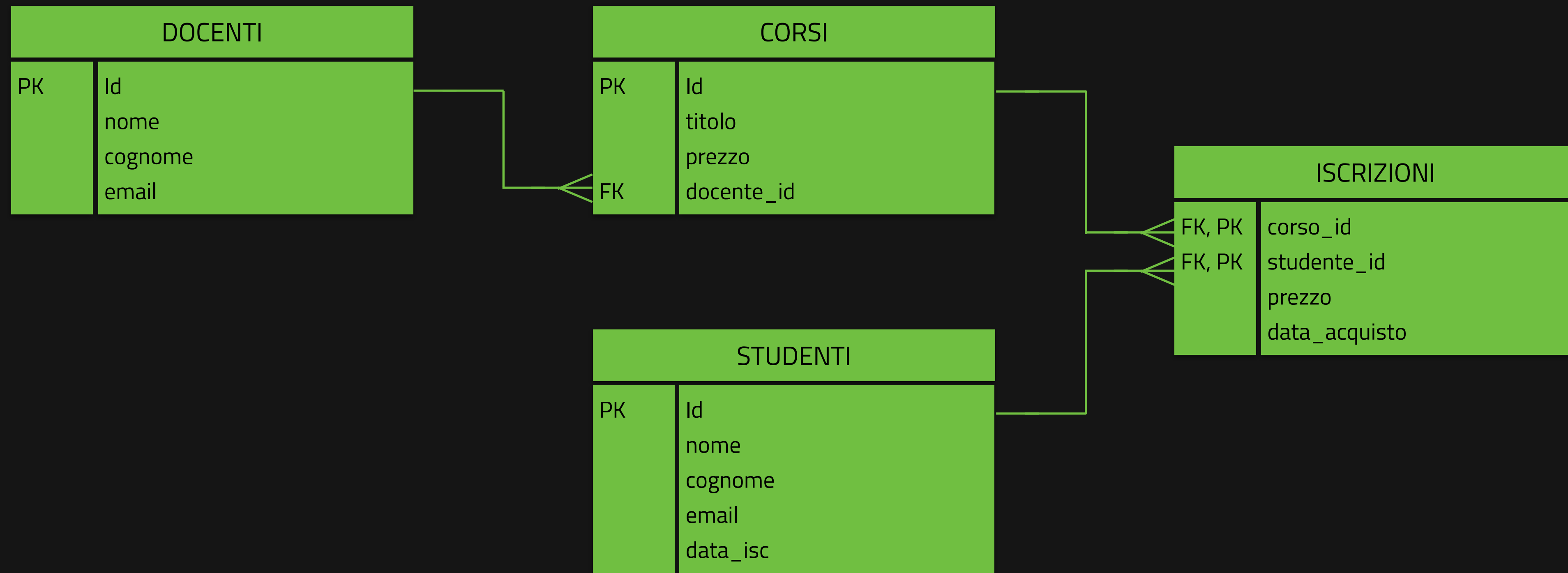
● id	titolo	prezzo	docente_id	● id	cognome	nome	email
1	Php	200.00	4	1	Rossi	Mario	rossi.mario@icloud.com
2	Javascript	120.00	2	2	Verdi	Paola	verdi.paola@gmail.com
3	Java	300.00	3	3	Bruni	Marco	bruni.marco@gmail.com
				4	Bianchi	Daniele	bianchi.daniele@icloud.com

Join tra tabelle

● id	titolo	prezzo	docente_id	● id	cognome	nome	email
1	Php	200.00	4	4	Bianchi	Daniele	bianchi.daniele@icloud.com
2	Javascript	120.00	2	2	Verdi	Paola	verdi.paola@gmail.com
3	Java	300.00	3	3	Bruni	Marco	bruni.marco@gmail.com

Esercizio: creare le tabelle relative al database "Corsi"

Allo schema fin qui disegnato manca la definizione dei domini per ogni attributo, bisogna cioè decidere il *data type*, eventuali *opzioni*, *gli indici*...



Riguardo alla tabella studenti possiamo partire da quella già creata, in modo da avere una anagrafica più completa (con più attributi rispetto allo schema qui sopra).

Dobbiamo avere solo cura di controllare e adeguare la struttura a questo database.

Per esempio modificando il nome dell'attributo *ins* in *data_isc*

Vediamo ora come interrogare più tabelle (*database corsi*):

Immaginiamo di voler estrarre l'elenco dei corsi e relativo docente assegnato.

Le informazioni richieste si trovano in due tabelle: *docenti* e *corsi*.

Le due tabelle sono in relazione *uno a molti* (un docente insegna in più corsi, un corso è insegnato da un solo docente).

La tabella *corsi* contiene l'attributo *docente_id* (chiave esterna) che la lega alla tabella *docenti*.

In *docente_id* viene infatti memorizzato l' *id* corrispondente al docente che insegna quel corso.

Vediamo la query:

```
SELECT nome, cognome, email, titolo  
FROM docenti, corsi  
WHERE docenti.id = corsi.docente_id;
```

Attraverso l'istruzione WHERE seleziono solo i record che hanno corrispondenza, quindi solo i corsi per i quali vi è associato un docente

Se volessimo estrarre l'elenco degli studenti e relativo corso a cui si sono iscritti dobbiamo interrogare tre tabelle per recuperare le informazioni necessarie:

il titolo si trova nella tabella *corsi*, il nome e il cognome degli studenti nella tabella *studenti*, l'iscrizione dello studente ad uno specifico corso nella tabella *iscrizioni* (associazione molti a molti). Vediamo la query:

```
SELECT nome, cognome, email, titolo  
FROM studenti, corsi, iscrizioni  
WHERE studenti.id = iscrizioni.studente_id  
AND corsi.id = iscrizioni.corso_id;
```

Attraverso l'istruzione WHERE seleziono solo i record che hanno corrispondenza per le condizioni indicate. L'operatore AND assicura che entrambe siano vere.

Vogliamo estrarre l'elenco completo delle informazioni sui corsi (*database corsi*).

Le informazioni richieste si trovano in quattro tabelle:

studenti, docenti, corsi e iscrizioni.

Vediamo la query:

```
SELECT studenti.nome, studenti.cognome, studenti.email, titolo, docenti.cognome,  
docenti.nome  
FROM studenti, corsi, iscrizioni, docenti  
WHERE studenti.id = iscrizioni.studente_id  
AND corsi.id = iscrizioni.corso_id  
AND docenti.id = corsi.docente_id;
```

Verranno estratti solo ed esclusivamente i valori che hanno una corrispondenza su tutte le tabelle.

Uso degli alias

Gli alias sono utilizzati per rinominare temporaneamente una tabella o un'intestazione di campo. Si usa l'istruzione **AS** (opzionale) quando si crea un alias¹.

- ◉ **Alias per le colonne:** è possibile utilizzare l'alias con GROUP BY, ORDER BY o HAVING per riferirsi alla colonna.

```
SELECT data_nascita AS `Data di nascita`  
FROM studente;
```

```
SELECT (campo1 - campo2) AS risultato  
FROM nome_tabella  
[WHERE condizione];
```

nota: AS si può anche omettere; SELECT data_nascita 'Data di nascita'...

¹⁾ se il nome alias è un nome composto (es: `Data di nascita`) o ha lettera accentata (es: `Età`) usate i backtick ` (apice retroverso) per wrappare il testo così da non avere risultati inattesi.

Approfondimento: <https://dev.mysql.com/doc/refman/8.0/en/problems-with-alias.html>

◉ Alias per le tabelle

Gli **alias per le tabelle** permettono di abbreviare i nomi delle tabelle, rendendo le query più concise e leggibili.

```
SELECT d.nome, d.cognome, d.email, c.titolo AS `Titolo Corso`  
FROM docenti AS d, corsi AS c  
WHERE d.id = c.docente_id  
ORDER BY `Titolo Corso`;
```

 Una volta assegnato un alias a una tabella, in tutta la query deve essere usato solo l'alias, non il nome originale della tabella.

Per migliorare la leggibilità, è consigliato sempre specificare l'alias della tabella davanti ai nomi delle colonne, anche quando non ci sono ambiguità.

Nel caso di colonne con lo stesso nome in più tabelle, l'uso dell'alias è obbligatorio per evitare errori e specificare chiaramente da quale tabella proviene il dato.

Vediamo la query che estrae l'elenco completo delle informazioni sui corsi con l'uso degli *alias*.

Le informazioni richieste si trovano in quattro tabelle:

studenti, docenti, corsi e iscrizioni.

Vediamo la query:

```
SELECT
  s.nome `Nome studente`,
  s.cognome `Cognome studente`,
  s.email `Contatto studente`,
  c.titolo `Iscritto a`,
  d.cognome `Cognome docente`,
  d.nome `Nome docente`
FROM studenti s, corsi c, iscrizioni i, docenti d
WHERE s.id = i.studente_id
AND c.id = i.corso_id
AND d.id = c.docente_id;
```


Integrità referenziale

FOREIGN KEY: gestire le relazioni tra tabelle

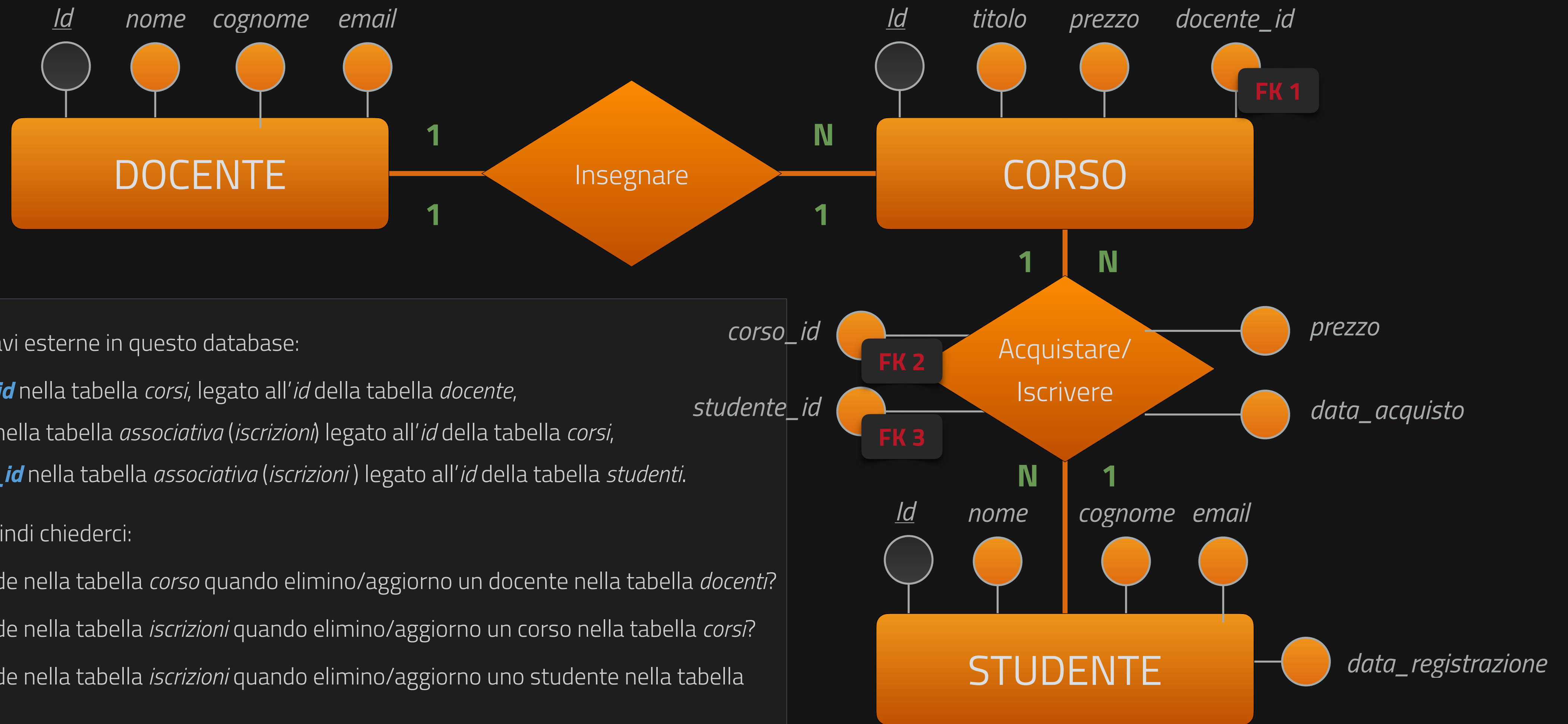
L'integrità referenziale è un insieme di regole che garantiscono la validità delle relazioni tra i record di tabelle correlate. Serve a prevenire la modifica o l'eliminazione accidentale di dati collegati.

Le chiavi esterne (Foreign Key) possono essere utilizzate solo se:

- ✓ Entrambe le tabelle utilizzano il motore InnoDB.
- ✓ Il campo della tabella primaria è una chiave primaria.
- ✓ I campi correlati hanno lo stesso tipo di dati.
- ✓ I campi coinvolti nella relazione hanno un indice (ad es. PRIMARY KEY, UNIQUE, o INDEX).
- ✓ Non si utilizzano campi di tipo BLOB o TEXT nelle chiavi coinvolte.

Le Foreign Key permettono di definire il comportamento del database quando si tenta di eliminare o modificare un record nella tabella primaria che è collegato a uno o più record nella tabella secondaria.

database "Corsi" - integrità dei dati



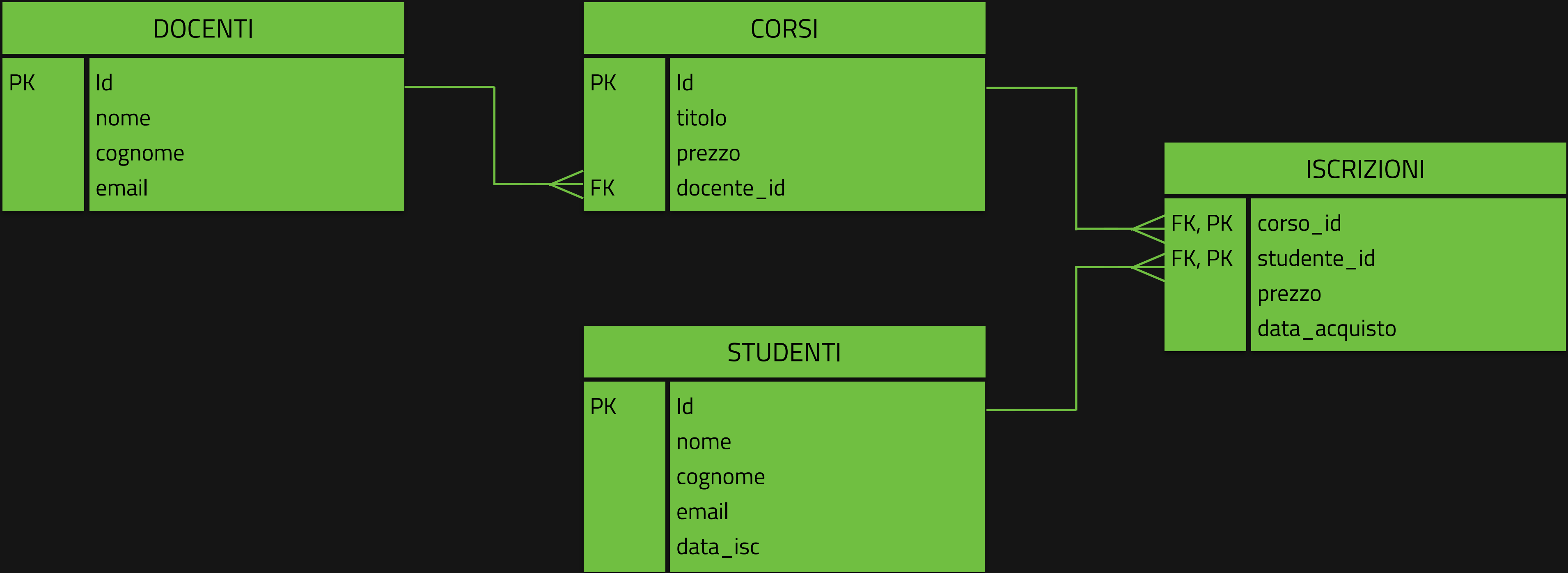
Ci sono 3 chiavi esterne in questo database:

- FK 1** **docente_id** nella tabella *corsi*, legato all'*id* della tabella *docente*,
- FK 2** **corso_id** nella tabella *associativa (iscrizioni)* legato all'*id* della tabella *corsi*,
- FK 3** **studente_id** nella tabella *associativa (iscrizioni)* legato all'*id* della tabella *studenti*.

Dobbiamo quindi chiederci:

1. Cosa succede nella tabella *corso* quando elimino/aggiorno un docente nella tabella *docenti*?
2. Cosa succede nella tabella *iscrizioni* quando elimino/aggiorno un corso nella tabella *corsi*?
3. Cosa succede nella tabella *iscrizioni* quando elimino/aggiorno uno studente nella tabella *studenti*?

tabelle relative al database "Corsi"



Possiamo definire alcune azioni diverse da far attivare in caso di cancellazione o modifica:

CASCADE

In questo caso la cancellazione o modifica di un record nella tabella primaria genererà la cancellazione o la modifica dei record collegati nella tabella secondaria

SET NULL

In caso di eliminazione o modifica di un record nella tabella primaria i record collegati della tabella secondaria verranno modificati impostando il valore del campo chiave esterna = NULL.

Questa azione è attivabile solo se il campo interessato della tabella secondaria non è impostato a NOT NULL (non deve essere required).

NO ACTION o RESTRICT

Queste due azioni (alternative) invece impediscono direttamente la modifica o la cancellazione dei record della tabella primaria. Praticamente specificare una di queste due azioni equivale a non eseguire alcuna azione.

Valore di default in assenza di indicazioni diverse quando si costruisce il vincolo di chiave esterna.

Vediamo un esempio. Creiamo un vincolo tra le due tabelle (*docenti*, *corsi*) per mezzo dei campi *docente.id* e *corsi.docente_id*

```
CREATE TABLE docenti (  
  id INT AUTO_INCREMENT,  
  nome VARCHAR(30),  
  cognome VARCHAR(50),  
  email VARCHAR(100),  
  PRIMARY KEY(id));  
  
CREATE TABLE corsi (  
  id INT AUTO_INCREMENT,  
  titolo VARCHAR(100),  
  prezzo DECIMAL(6,2),  
  docente_id INT,  
  PRIMARY KEY (id),  
  INDEX docente_key(docente_id), -- viene aggiunto dal motore quando creiamo la chiave esterna  
  CONSTRAINT fk_corsi_docenti  
    FOREIGN KEY(docente_id) REFERENCES docenti(id)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION); -- possiamo anche omettere le azioni in questo caso perché RESTRICT o NO ACTION sono default
```

In questo modo

- abbiamo creato una chiave esterna nella tabella secondaria *corsi* riferita al campo *id* della tabella primaria *docenti* chiamata *fk_corsi_docenti*;
- abbiamo impostato le azioni da seguire in caso di eliminazione o aggiornamento di un record nella tabella editore.

Nell'esempio abbiamo stabilito che non possiamo eliminare un docente se il suo *id* è presente nella tabella *corsi*.

```
DELETE FROM docenti  
WHERE id = 1;
```

Questa query restituisce l'errore:

```
Cannot delete or update a parent row: a foreign key...
```

Nel caso specifico abbiamo stabilito che non si può eliminare un docente dalla tabella "docenti" se il suo id è presente nella tabella corsi, prima bisogna eliminare le dipendenze nella tabella corsi.

```
DELETE FROM corsi  
WHERE docente_id=1;
```

Ora possiamo eseguire la query precedente:

```
DELETE FROM docenti  
WHERE id = 1;
```

Se volessimo modificare la FOREIGN KEY dovremmo prima eliminarla:

```
ALTER TABLE corsi  
DROP FOREIGN KEY fk_corsi_docenti;
```

e poi ricrearla da capo con le nuove regole:

```
ALTER TABLE corsi ADD CONSTRAINT fk_corsi_docenti  
FOREIGN KEY(docente_id) REFERENCES docenti(id)  
ON DELETE CASCADE ON UPDATE CASCADE;
```

In questo modo abbiamo ricreato la chiave esterna con le nuove azioni da seguire in caso di eliminazione o aggiornamento di un record nella tabella *docenti*.

Ora la query seguente elimina a cascata tutti i record associati nella tabella figlia (*corsi*)

```
DELETE FROM docenti  
WHERE id = 1;
```



```
ALTER TABLE corsi  
DROP FOREIGN KEY fk_corsi_docenti;
```

nuove regole:

```
ALTER TABLE corsi ADD CONSTRAINT fk_corsi_docenti  
FOREIGN KEY(docente_id) REFERENCES docenti(id)  
ON DELETE SET NULL ON UPDATE CASCADE;
```

Nel caso specifico abbiamo stabilito che eliminando un docente dalla tabella “docenti”, impostiamo a *NULL* le dipendenze (campo *docente_id*) nella tabella *corsi*.

```
DELETE FROM docenti  
WHERE id = 1;
```

Questa query viene eseguita; contemporaneamente vengono aggiornate le righe con il campo *docente_id* = 1 della tabella *corsi*, impostando il valore del campo *docente_id* a *NULL*

Possiamo definire anche una chiave esterna *referita alla stessa tabella*.

Possiamo definire cioè una SELF-FOREIGN KEY.

Consideriamo come esempio una ipotetica tabella "impiegati" dove per ciascun impiegato registro anche l'attributo identificativo del suo impiegato responsabile:

```
CREATE TABLE IF NOT EXISTS impiegati (  
  id int auto_increment,  
  nome varchar(50),  
  cognome varchar(50),  
  ruolo varchar(50),  
  id_responsabile int,  
  stipendio decimal(6,2),  
  FOREIGN KEY impiegati(id_responsabile) REFERENCES impiegati(id)  
  ON DELETE SET NULL,  
  PRIMARY KEY(id)  
);
```

In questo caso il campo *id_responsabile* è chiave esterna riferita all' *id* di ciascun impiegato.

Per visualizzare la chiave esterna di una tabella:

```
SHOW CREATE TABLE nome_tabella;
```

Per visualizzare tutte le chiavi esterne del db:

```
SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME  
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE  
WHERE TABLE_SCHEMA = 'nome_db'  
/* AND TABLE_NAME = 'nome_tabella' */  
AND referenced_column_name IS NOT NULL;
```

Per disabilitare* temporaneamente le chiavi esterne:

```
SET FOREIGN_KEY_CHECKS=0;
```

Ricordatevi di ripristinarle dopo un eventuale inserimento massiccio di record

```
SET FOREIGN_KEY_CHECKS=1;
```

* disabilitare/abilitare le chiavi esterne può servire a popolare un db senza tenere conto dell'ordine di inserimento dei dati che altrimenti verrebbe controllato dalle chiavi esterne - FOREIGN_KEY_CHECKS è una variabile di sistema;

es: con chiave esterna attiva non posso caricare record dei corsi prima di caricare i record degli docenti

CHECK (constraint)

I vincoli forniscono restrizioni sui dati che è possibile aggiungere a una tabella. Ciò consente di applicare il controllo dei dati da MySQL, piuttosto che tramite la logica dell'applicazione.

Quando un'istruzione viola un vincolo, MySQL genera un errore.

Esistono quattro tipi di vincoli di tabella:

- **PRIMARY KEY**: Imposta la colonna per fare riferimento alle righe. I valori devono essere univoci e non nulli.
- **FOREIGN KEY**: Imposta la colonna in modo che faccia riferimento alla chiave primaria su un'altra tabella.
- **UNIQUE**: Richiede valori nella colonna o nelle colonne si verificano solo una volta nella tabella.
- **CHECK**: Verifica se i dati soddisfano la condizione specificata.

CHECK

A partire da MySQL 8.0.16 vengono applicati i *CHECK*.

Prima di MySQL 8.0.16 le espressioni di vincolo erano accettate nella sintassi ma ignorate.

In MYSQL puoi definire i vincoli in 2 modi diversi:

1. *CHECK (expression)* dato come parte di una definizione di colonna;
2. *CONSTRAINT [constraint_name] CHECK (expression)* come vincolo di tabella (può riferirsi a più colonne).

Prima che una riga venga inserita o aggiornata, tutti i vincoli vengono valutati nell'ordine in cui sono definiti. Se un'espressione di vincolo restituisce false, la riga non verrà inserita o aggiornata. È possibile utilizzare la maggior parte delle funzioni deterministiche in un vincolo, comprese le UDF (User Defined Function).

Se non dai un nome al vincolo, il vincolo riceverà un nome generato automaticamente.

In modo da poter successivamente eliminare il vincolo con:

```
ALTER TABLE nome_tabella DROP CONSTRAINT nome_vincolo;
```


Uso CHECK alla creazione della tabella

```
CREATE TABLE libri2 (  
    id int AUTO_INCREMENT,  
    titolo varchar(255),  
    prezzo decimal(6,2) NOT NULL,  
    pagine smallint unsigned CHECK (pagine > 0) /* CONSTRAINT chk_pagine CHECK (pagine > 0) */,  
    editore_id int,  
    PRIMARY KEY (id),  
    /* CHECK su due colonne */  
    CONSTRAINT chk_prezzo_pagine CHECK ((prezzo > 0) AND (pagine > 0))  
);
```

Aggiunta CHECK su tabella esistente con nome definito dall'utente:

```
ALTER TABLE studenti  
ADD CONSTRAINT ck_eta CHECK(eta >= 18);
```

```
ALTER TABLE libri  
ADD CONSTRAINT ck_prezzo CHECK(prezzo > 0);
```

Aggiunta CHECK su tabella esistente con nome definito dal motore ([nome_tabella]_chk_[numero sequenziale 1,2,...])

```
ALTER TABLE studenti  
ADD CHECK(eta >= 18);
```

```
ALTER TABLE libri  
ADD CHECK(prezzo > 0);
```

Visualizzare le CONSTRAINT CHECK con:

SHOW CREATE TABLE *nome_tabella*

```
SHOW CREATE TABLE libri2;
```

Visualizzare le CONSTRAINT CHECK interrogando
information_schema:

```
SELECT *  
FROM information_schema.table_constraints  
WHERE constraint_type = 'CHECK'  
      AND table_name = 'libri';
```

Unire le tabelle: UNION

L'operatore di MySQL **UNION** consente di combinare due o più set di risultati da più tabelle in un singolo set di risultati. La sintassi è la seguente:

```
SELECT COLUMN1, COLUMN2 FROM table1
UNION [DISTINCT | ALL]
SELECT COLUMN1, COLUMN2 FROM table2
UNION [DISTINCT | ALL]
SELECT COLUMN1, COLUMN2 FROM table3;
```

Affinché una **UNION** funzioni correttamente è necessario che:

- il *numero delle colonne selezionate sia uguale* in ciascuna delle query che si desidera unire;

Per impostazione predefinita, l'operatore **UNION** elimina le righe duplicate dal risultato anche se non si utilizza esplicitamente l'operatore DISTINCT. Pertanto, si dice che la clausola **UNION** è la scorciatoia dell'UNION DISTINCT.

Se si utilizza **UNION ALL** esplicitamente, le righe duplicate, se disponibili, vengono mostrate.

UNION ALL si esegue più velocemente di UNION DISTINCT.

Vediamo un esempio:

```
SELECT cognome, data_nascita, 'X' Generazione
FROM studenti
WHERE data_nascita <= '1980-12-31'

UNION ALL

SELECT cognome, data_nascita, 'Millenials' Generazione
FROM studenti
WHERE data_nascita BETWEEN '1981-01-01' AND '1996-12-31'

UNION ALL

SELECT cognome, data_nascita, 'Z' Generazione
FROM studenti
WHERE data_nascita >= '1997-01-01'

ORDER BY data_nascita;
```

otteniamo l'elenco degli studenti divisi per generazione

Altro esempio:

```
SELECT stato, capitale FROM europa  
UNION ALL  
SELECT stato, capitale FROM africa  
UNION ALL  
SELECT stato, capitale FROM americhe  
ORDER BY stato;
```

Otteniamo l'elenco degli stati e delle capitali dei tre continenti

Altro esempio:

```
SELECT nome, cognome FROM parente  
UNION ALL  
SELECT nome, cognome FROM studente  
ORDER BY cognome;
```



```
TABLE table1  
UNION [DISTINCT | ALL]  
SELECT COLUMN1, COLUMN2 FROM table2  
UNION [DISTINCT | ALL]  
TABLE table3;
```

Al posto della select si può usare anche TABLE o un mix delle istruzioni:

```
TABLE europa  
UNION ALL  
SELECT id, stato, capitale FROM africa  
UNION ALL  
TABLE americhe  
ORDER BY stato;
```

INTERSECT e EXCEPT

MySQL supporta gli operatori **INTERSECT** ed **EXCEPT** a partire da MySQL 8.0.31.

INTERSECT.

La query restituisce *solo* i valori comuni alle due tabelle:

```
SELECT * FROM amici  
INTERSECT [DISTINCT | ALL]  
SELECT * FROM parenti;
```

EXCEPT.

La query restituisce i valori della prima tabella che non sono presenti nella seconda:

```
SELECT * FROM amici  
EXCEPT [DISTINCT | ALL]  
SELECT * FROM parenti;
```

EQUIJOIN o JOIN semplice

La query per mostrare l'elenco dei corsi e relativo docente è un esempio di JOIN semplice.

La relazione tra le tabelle è data dall'uguaglianza della *chiave* della tabella padre e della *chiave esterna* della tabella figlia.

La query restituisce *solo* i valori corrispondenti. se una riga non soddisfa la condizione non verrà mostrata:

```
SELECT corsi.id, titolo, prezzo, docente_id, docenti.id, nome, email
FROM docenti, corsi
WHERE docenti.id = corsi.docente_id;
```

● id	titolo	prezzo	docente_id	● id	cognome	nome	email
1	Php	200.00	4	4	Bianchi	Daniele	bianchi.daniele@icloud.com
2	Javascript	120.00	2	2	Verdi	Paola	verdi.paola@gmail.com
3	Java	300.00	3	3	Bruni	Marco	bruni.marco@gmail.com

Unire le tabelle, sintassi: INNER JOIN

JOIN è un costrutto del linguaggio SQL attraverso il quale vengono messe in relazione due tabelle, ed è alternativo al costrutto precedente che usa WHERE

Lo scopo di JOIN è quello di unire le tabelle restituendo un risultato combinato sulla base di uno o più campi che trovano corrispondenza in tutte le tabelle coinvolte nella JOIN.

Il collegamento tra le tabelle viene effettuato mediante **INNER JOIN** e la relazione viene stabilita mediante la clausola **ON** che identifica i campi che, nelle tabelle, devono offrire l'eguaglianza: *verranno estratti, infatti, solo ed esclusivamente i valori che hanno una corrispondenza su tutte le tabelle.*

Esempio di sintassi:

```
SELECT tabella1.campo1, tabella2.campo2  
FROM tabella1  
INNER JOIN tabella2 ON tabella1.key=tabella2.key
```

JOIN su più tabelle

Il join fra n tabelle implica un minimo di $n-1$ condizioni di join.

Per esempio, un join tra 3 tabelle implica almeno 2 condizioni di join.

Per le tabelle a, b, c :

Le condizioni di join saranno 2:

```
SELECT *  
FROM a, b, c  
WHERE a.key=b.key  
AND b.key=c.key;
```

con la sintassi JOIN

```
SELECT *  
FROM a  
[INNER] JOIN b ON a.key=b.key  
[INNER] JOIN c ON b.key=c.key;
```

Per esempio per estrarre i corsi e i relativi studenti iscritti le tabelle da interrogare sono tre.

Provate a costruire la query...

INNER JOIN

Vogliamo estrarre l'elenco dei corsi e relativo docente usando il costrutto INNER JOIN (INNER possiamo ometterlo).

Vediamo la query:

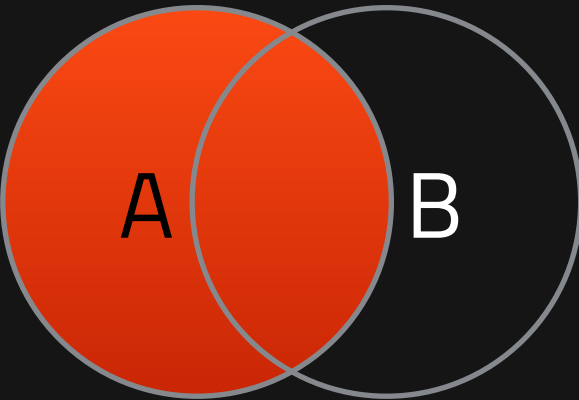
```
SELECT titolo AS Corso, cognome AS Cognome, nome AS Nome, email  
FROM docenti  
JOIN corsi  
ON corsi.docente_id = docente.id;
```

Vogliamo estrarre l'elenco degli studenti e relativo corso usando il costrutto INNER JOIN.

```
SELECT cognome AS Cognome, nome AS Nome, email, titolo AS Corso  
FROM studenti s  
JOIN iscrizioni i ON s.id = i.studente_id  
JOIN corsi c ON c.id = i.corso_id;
```

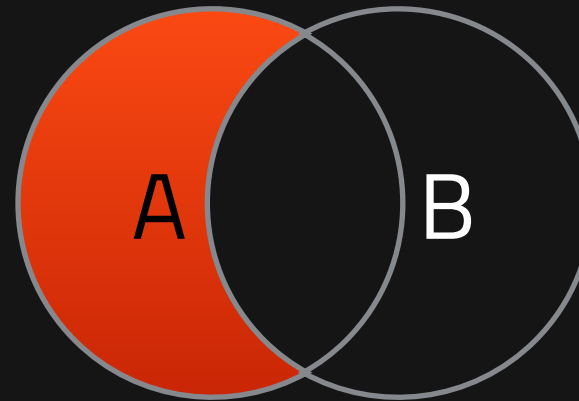

LEFT JOIN

Estrae tutti i valori della tabella a sinistra anche se non hanno corrispondenza nella tabella a destra



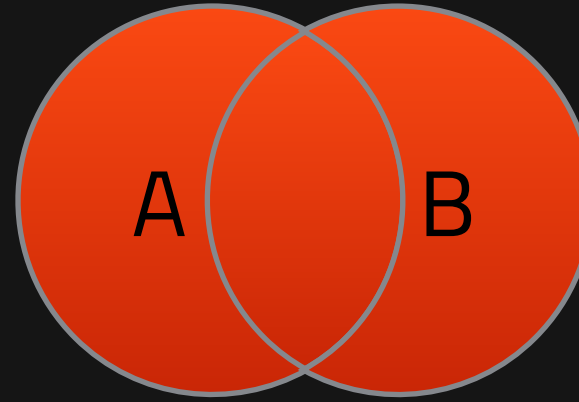
```
SELECT list
FROM tableA
LEFT JOIN tableB
ON A.key=B.key
```

Estrae solo i valori della tabella a sinistra che non hanno corrispondenza nella tabella a destra



```
SELECT list
FROM tableA
LEFT JOIN tableB
ON A.key=B.key
WHERE B.key
IS NULL
```

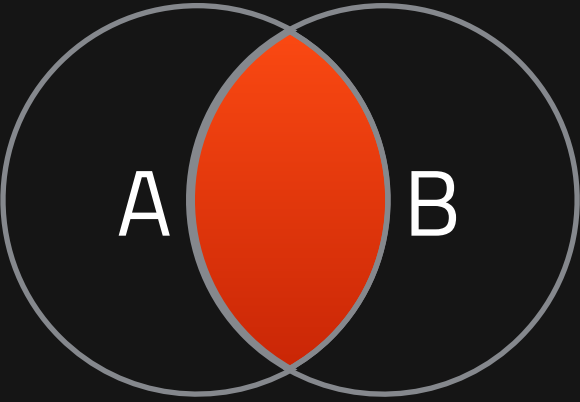
Estrae sia valori che hanno una corrispondenza su tutte le tabelle sia quelli che non ce l'hanno



```
SELECT list
FROM tableA
FULL OUTER JOIN tableB
ON A.key=B.key
```

INNER JOIN

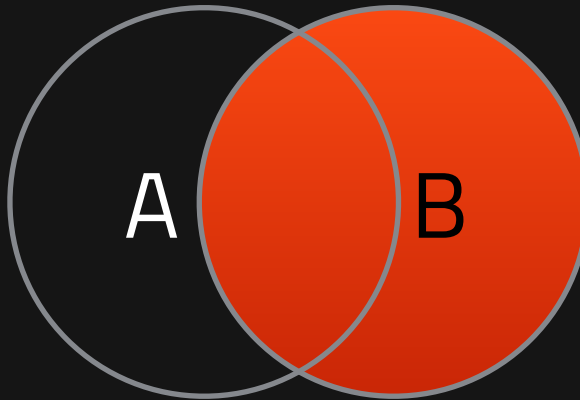
Estrae esclusivamente i valori che hanno una corrispondenza su tutte le tabelle



```
SELECT list
FROM tableA
INNER JOIN tableB
ON A.key=B.key
```

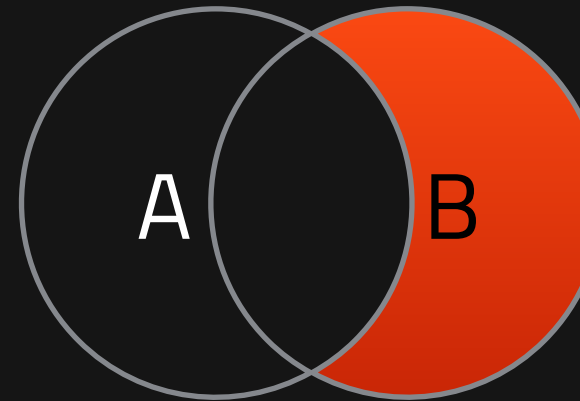
RIGHT JOIN

Estrae tutti i valori della tabella a destra anche se non hanno corrispondenza nella tabella a sinistra



```
SELECT list
FROM tableA
RIGHT JOIN tableB
ON A.key=B.key
```

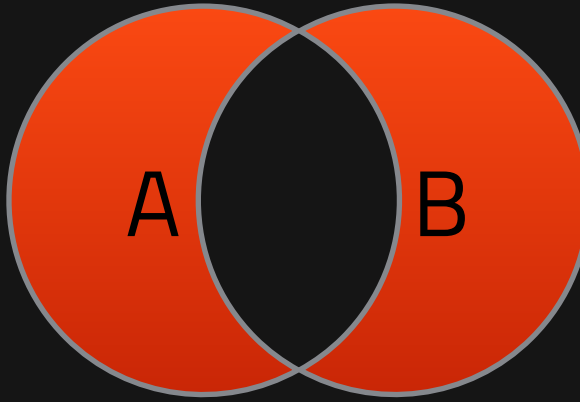
Estrae solo i valori della tabella a destra che non hanno corrispondenza nella tabella a sinistra



```
SELECT list
FROM tableA
RIGHT JOIN tableB
ON A.key=B.key
WHERE A.key
IS NULL
```

FULL OUTER JOIN

Estrae esclusivamente i valori che non hanno una corrispondenza su tutte le tabelle



```
SELECT list
FROM tableA
FULL OUTER JOIN tableB
ON A.key=B.key
WHERE A.key IS NULL
OR B.key IS NULL
```


OUTER JOIN

A differenza delle INNER JOIN, le **OUTER JOIN** selezionano i risultati anche in assenza di una corrispondenza su entrambe le tabelle.

Più precisamente è possibile definire in MySQL due tipi di OUTER JOIN, cioè:

- **LEFT JOIN**: estrae tutti i valori della tabella a sinistra anche se non hanno corrispondenza nella tabella a destra;
- **RIGHT JOIN**: estrae tutti i valori della tabella a destra anche se non hanno corrispondenza nella tabella di sinistra.

Tabella di *sinistra* e tabella di *destra* si riferiscono all'ordine con cui compaiono le tabelle nella query.

OUTER JOIN

Vogliamo estrarre l'elenco dei docenti considerando anche quelli a cui non è stato associato un corso.

Usiamo **LEFT JOIN** per estrarre anche i record dalla tabella di sinistra (docenti -la prima che compare nella query) che non hanno corrispondenza con la tabella di destra (corsi).

Vediamo la query:

```
SELECT cognome, nome, email, titolo AS corso
FROM docenti d
LEFT JOIN corsi c
ON d.id = c.docente_id;
```

Usando **RIGHT JOIN** otteniamo anche i record della tabella di destra(corsi) che non hanno corrispondenza con la tabella di sinistra (docenti).

Vediamo la query:

```
SELECT titolo AS corso, cognome, nome, email
FROM docenti d
RIGHT JOIN corsi c
ON d.id = c.docente_id;
```

Vogliamo estrarre solo l'elenco dei docenti a cui non è stato assegnato alcun corso.

Usiamo sempre LEFT JOIN per estrarre i record dalla tabella di sinistra (docenti- la prima che compare nella query) che non hanno corrispondenza con la tabella di destra.

Tra questi selezioniamo solo quelli per cui il valore del campo chiave della tabella B risulta NULL (vedi diagramma di Venn).

Vediamo la query:

```
SELECT cognome, nome, email, titolo AS corso
FROM docenti d
LEFT JOIN corsi c
ON d.id = c.docente_id
WHERE c.id IS NULL;
```

Vogliamo estrarre l'elenco solo dei corsi a cui non è stato associato alcun docente.

Vediamo la query:

```
SELECT titolo AS corso, cognome, nome, email
FROM docenti d
RIGHT JOIN corsi c
ON d.id = c.docente_id
WHERE d.id IS NULL;
```

FULL OUTER JOIN

In mysql non sono implementate le *FULL OUTER JOIN*.

Ma con un escamotage si possono ottenere ugualmente:
attraverso l'unione (UNION) di due *OUTER JOIN: LEFT* e *RIGHT*.

```
SELECT cognome, nome, email, titolo AS corso  
FROM docenti d  
LEFT JOIN corsi c  
ON d.id = c.docente_id
```

UNION

```
SELECT cognome, nome, email, titolo AS corso  
FROM docenti d  
RIGHT JOIN corsi c  
ON d.id = c.docente_id;
```


FULL OUTER JOIN

solo gli esclusi

```
SELECT cognome, nome, email, titolo AS corso
FROM docenti d
LEFT JOIN corsi c
ON d.id = c.docente_id
WHERE c.id IS NULL
```

UNION

```
SELECT cognome, nome, email, titolo AS corso
FROM docenti d
RIGHT JOIN corsi c
ON d.id = c.docente_id
WHERE d.id IS NULL;
```

SELF JOIN

Immaginiamo di avere una tabella "impiegato".

All'interno della tabella registriamo, tra gli attributi (*nome, cognome, ruolo, id_resp, stipendio, ufficio_id*), anche l'id del responsabile di ciascun impiegato in *id_resp*.

Se volessimo conoscere l'elenco degli impiegati e il loro responsabile possiamo usare una SELF JOIN grazie al meccanismo degli alias.

Vediamo la query:

```
SELECT i.cognome, i.nome, i.ruolo, r.cognome Responsabile  
FROM impiegati i  
JOIN impiegati r  
ON i.id_responsabile = r.id  
ORDER BY ruolo;
```

Vediamo la query che comprende anche gli impiegati responsabili:

```
SELECT i.cognome, i.nome, i.ruolo, r.cognome Responsabile  
FROM impiegati i  
LEFT JOIN impiegati r  
ON i.id_responsabile = r.id  
ORDER BY ruolo;
```

Scorciatoia JOIN

Se abbiamo usato la stessa label per i *campi chiave* e la *chiave esterna* (es: *docente_id* nella tabella *docente* ed *docente_id* nella tabella *corsi*) possiamo scrivere la join usando l'istruzione USING¹ in questo modo:

```
SELECT cognome, nome, email, titolo AS corso  
FROM docenti  
JOIN corsi  
USING (docente_id);
```

¹La clausola nomina un elenco di colonne che devono esistere in entrambe le tabelle.

Non EQUI-JOIN

Un join non equi-join è un tipo di join che utilizza una condizione diversa dall'uguaglianza (come <, <=, >, >=, !=, ecc.) per combinare le righe di due tabelle.

Vediamo un esempio utilizzando due tabelle: *studenti* e *generazioni*.

La tabella *generazioni* [da creare nel nostro database] definisce gli intervalli degli anni che comprendono una specifica generazione:

- Boomers (dal 1946 al 1964)
- Generazione X (dal 1965 al 1980)
- Millenials (dal 1981 al 1996)
- Generazione Z (dal 1997 al 2012)

Questa tabella viene definita **tabella di lookup**.

Una *tabella di lookup* è una tabella che contiene un insieme di valori statici o semi-statici utilizzati per classificare o categorizzare dati in altre tabelle.

In pratica, viene utilizzata per:

- Mantenere valori di riferimento (come i nomi delle generazioni),
- Normalizzare il database, riducendo la ridondanza dei dati,
- Migliorare l'efficienza e la chiarezza delle query, mantenendo un unico punto di riferimento.

Vogliamo associare ogni studente alla sua generazione basata sul suo anno di nascita.

Questo richiede un non equi-join.

Di seguito due esempi identici con operatori diversi.

```
SELECT s.cognome, s.nome, s.data_nascita, g.generazione
FROM studenti s
JOIN generazioni g
ON s.data_nascita >= g.anno_inizio
WHERE s.data_nascita <= g.anno_fine
ORDER BY s.data_nascita;
```

```
SELECT s.cognome, s.nome, s.data_nascita, g.generazione
FROM studenti s
JOIN generazioni g
ON s.data_nascita
BETWEEN g.anno_inizio AND g.anno_fine
ORDER BY s.data_nascita;
```


Funzioni

Funzioni di aggregazione

Possiamo eseguire calcoli su tutta la colonna utilizzando le funzioni di aggregazione di SQL.

Questo ci permetterà di guardare i dati nel loro insieme e ottenere calcoli come la media, il numero di valori, la somma totale di tutti i valori e altro ancora.

AVG() non considera i valori nulli

La funzione *AVG()* restituisce il valore medio (media) dell'intero set di dati.

```
SELECT AVG(campo)
FROM tabella;
```

```
SELECT AVG(prezzo) AS 'prezzo medio'
FROM corsi;
```

COUNT() considera i valori nulli, COUNT[(colonna)] non considera i valori nulli*

La funzione *COUNT()* restituisce il numero di record trovati.

```
SELECT COUNT(*)  
FROM tabella;
```

Possiamo anche filtrare il conteggio, vengono contati i record che soddisfano la condizione:

```
SELECT COUNT(*)  
FROM tabella  
WHERE condizione;
```

Per contare il numero delle studentesse donna:

```
SELECT COUNT(*)  
FROM studenti  
WHERE genere='f';
```

Per contare il numero degli studenti a cui non è stato assegnato il valore per l'attributo 'genere':

```
SELECT COUNT(*)  
FROM studenti  
WHERE genere IS NULL;
```

MAX() e MIN() non considera i valori nulli

Le funzioni MAX() e MIN() restituiscono il valore più alto e il valore più basso del campo richiesto nella query.

```
SELECT MAX(campo), MIN(campo)
FROM tabella;
```

```
SELECT MAX(prezzo) AS 'più caro', MIN(prezzo) AS 'più economico'
FROM corsi;
```

SUM() non considera i valori nulli

La funzione SUM() restituisce la somma dei valori di un dato campo.

```
SELECT SUM(campo)
FROM tabella;
```

```
SELECT SUM(prezzo) AS 'Valore corsi'
FROM corsi;
```


Funzioni matematiche

FLOOR() e CEILING()

Le funzioni FLOOR() e CEILING() arrotondano rispettivamente verso il basso e verso l'alto i numeri dopo la virgola.

```
SELECT FLOOR(campo), CEILING(campo)
FROM tabella;
```

```
SELECT FLOOR(prezzo), CEILING(prezzo)
FROM libri;
```

ROUND()

La funzione ROUND() rispetta l'arrotondamento matematico.

Accetta un secondo argomento in cui indicare i decimali dopo la virgola.

```
SELECT ROUND(campo)
FROM tabella;
```

```
SELECT ROUND(prezzo,2)
FROM libri;
```

Funzioni sulle stringhe

LENGTH()

La funzione LENGTH() restituisce il numero totale di caratteri in ogni campo. Anche gli spazi tra le parole sono contati

```
SELECT nome, LENGTH(nome)  
FROM studenti;
```

CONCAT(campo1, campo2, campo3): concatena le stringhe;

```
SELECT CONCAT(nome, ' ', cognome)  
FROM studenti;
```

La funzione CONCAT_WS () concatena due o più espressioni insieme e aggiunge un separatore tra di esse.

```
SELECT CONCAT_WS(' ', nome, cognome)  
FROM studenti;
```

SUBSTRING(str, pos, len) o SUBSTRING(str FROM pos FOR len);

se non specifichiamo "len" vengono presi tutti i caratteri seguenti alla posizione indicata

```
SELECT nome, SUBSTRING(nome, 2, 3)
FROM studenti;
```

LEFT() / RIGHT():

mostra uno specifico numero di caratteri iniziando da sinistra(LEFT) o destra(RIGHT)

```
SELECT nome, LEFT(nome, 1)
FROM studenti;
```

```
SELECT CONCAT(LEFT(nome, 1), '.', cognome)
FROM studenti;
```


Uso di funzioni combinate

Lunghezza media in termini di caratteri considerando la stringa *nome* + *cognome* insieme:

```
SELECT AVG(LENGTH(nome)+LENGTH(cognome))  
FROM studenti;
```

```
SELECT AVG(LENGTH(CONCAT(nome, cognome)))  
FROM studenti;
```

Elimina i numeri dopo la virgola, FLOOR() arrotonda verso il basso, CEILING() verso l'alto;

```
SELECT FLOOR(AVG(LENGTH(CONCAT(nome, cognome))))  
FROM studenti;
```

```
SELECT CEILING(AVG(LENGTH(CONCAT(nome, cognome))))  
FROM studenti;
```

Funzioni informative¹

LAST_INSERT_ID().

LAST_INSERT_ID() restituisce un valore (a 64 bit) BIGINT UNSIGNED che rappresenta il primo valore generato automaticamente inserito correttamente per una colonna AUTO_INCREMENT come risultato dell'ultima istruzione INSERT eseguita.

Il valore di LAST_INSERT_ID() rimane invariato se nessuna riga viene inserita correttamente.

Dopo aver inserito una riga che genera un AUTO_INCREMENT

```
INSERT INTO studenti(cognome, nome, email)
VALUE('rossi', 'marco', 'marco.rossi@gmail.com');
```

si può ottenere il valore in questo modo:

```
SELECT LAST_INSERT_ID();
-- 28 valore dell'auto increment inserito (il valore riportato è un esempio)
-- se si inseriscono più record viene sempre restituito il primo id inserito del gruppo
```

¹) <https://dev.mysql.com/doc/refman/8.0/en/information-functions.html>

Aggiornamento / sostituzione

Se dobbiamo aggiornare parte del testo di un campo nella nostra tabella possiamo usare una delle funzioni stringa:

REPLACE

Vediamo ad esempio come aggiornare le informazioni presenti in un campo modificando solo una parte del suo valore

```
UPDATE studenti  
SET email = REPLACE(email, '.com', '.it');
```

La funzione utilizza tre argomenti racchiusi tra parentesi:

- il primo argomento è il campo di riferimento per la ricerca/sostituzione;
- il secondo argomento, racchiuso tra apici, rappresenta la stringa di testo da sostituire;
- il terzo argomento, racchiuso tra apici, rappresenta la stringa nuova.

Possiamo usarla anche in lettura se vogliamo sostituire al volo una sotto-stringa:

```
SELECT email, REPLACE(email, '.com', '.it')  
FROM studenti;
```

funzioni data e ora

per impostare le informazioni temporali attuali, si possono usare le seguenti funzioni:

FUNZIONE	DESCRIZIONE
NOW()	restituisce data e ora attuali. Ammette i sinonimi CURRENT_TIMESTAMP() e CURRENT_TIMESTAMP
CURDATE()	restituisce data attuale. Ammette i sinonimi CURRENT_DATE() e CURRENT_DATE
CURTIME()	restituisce orario attuale. Ammette i sinonimi CURRENT_TIME() e CURRENT_TIME

Una volta impostate, le informazioni data/ora possono essere lette, in tutto o in parte, prelevandone solo alcuni elementi. Per queste operazioni, esistono apposite funzioni:

- giorni, mesi e anni: YEAR(), MONTH() e DAY() che, rispettivamente, restituiscono anno, mese e giorno;
- ore, minuti e secondi: HOUR(), MINUTE() e SECOND();
- giorno nella settimana o nell’anno: DAYOFWEEK() (restituisce: da 1 a 7); DAYOFYEAR() (restituisce: da 1 a 365)
- nome del giorno della settimana DAYNAME(), nome del mese MONTHNAME()

```
SELECT YEAR(CURDATE());
```

```
SELECT MONTH(CURDATE());
```

```
SELECT DAY(CURDATE());
```

```
SELECT HOUR(CURTIME());
```

```
SELECT MINUTE(CURTIME());
```

```
SELECT SECOND(CURTIME());
```

```
SELECT DAYNAME(CURDATE());
```

```
SELECT DAYNAME('vostra_data_nascita'); #se volete sapere il giorno i cui siete nati
```

```
SELECT MONTHNAME(CURDATE());
```

Per avere i nomi in lingua bisogna impostare la lingua italiana (potreste non avere i privilegi):

```
SET LC_TIME_NAMES='it_IT'; #(https://dev.mysql.com/doc/refman/5.7/en/locale-support.html)
```

Per conoscere la lingua utilizzata: SELECT @@lc_time_names;

```
SELECT DAYOFWEEK(CURDATE()); #notate il numero restituito pur avendo impostato la lingua italiana
```

```
SELECT DAYOFMONTH(CURDATE());
```

```
SELECT DAYOFYEAR(CURDATE());
```

```
SELECT WEEKOFYEAR(CURDATE());
```


Formattare date e orari

DATE_FORMAT() e TIME_FORMAT().

La funzione DATE_FORMAT() permette di personalizzare il formato di una data usando appositi meta-caratteri:

METACARATTERE	DESCRIZIONE
%d	giorno del mese, %D numerale ordinale del giorno
%m	mese espresso in numero. La variante %M esprime il mese in parole
%Y	l'anno su quattro cifre. La variante %y esprime l'anno su due cifre
%H	indica le ore (da 0 a 24, in alternativa %h le mostra da 0 a 12)
%i	indica i minuti
%s	%S o %s per i secondi.
%p	indica AM o PM

NOTA: si applicano ad altre funzioni: STR_TO_DATE(), TIME_FORMAT(), UNIX_TIMESTAMP()

Elenco completo: https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html#function_date-format

```
SELECT DATE_FORMAT('2017-02-28','%d/%m/%Y'); #restituisce: 28/02/2017
```

```
SELECT DATE_FORMAT('2017-02-28','%d/%m/%y'); #restituisce: 28/02/17
```

```
SELECT DATE_FORMAT('2017-02-28','%d %M %Y'); #restituisce: 28 Febbraio 2017
```

```
SELECT TIME_FORMAT('17:25:34','%H-%i'); #restituisce: 17-25
```

```
SELECT TIME_FORMAT('17:25:34','%h:%i %p'); #restituisce: 05:25 PM
```

```
SELECT TIME_FORMAT(CURTIME(),'sono le %H e %i minuti') AS 'che ore sono?';  
#restituisce: 'sono le 18 e 58 minuti'
```

Se interrogate una tabella la formattazione della data può essere:

```
SELECT nome, cognome, DATE_FORMAT(data_nascita,'%d %M %Y')  
FROM studenti;
```

il formato data impostato è tra apici, è quindi una stringa in cui potete inserire anche del testo: vedi esempio TIME_FORMAT().

STR_TO_DATE(str, format)

Prende una stringa `STR['01-01-2001']` e un formato `FORMAT[%d-%m-%Y]` .

`STR_TO_DATE()` restituisce sia un valore DATETIME, se specifichiamo nel formato data e ora, sia un valore DATE o TIME, se specifichiamo solo data o solo ora.

Se la data, l'ora o il valore DATETIME estratti da *str* è illegale, `STR_TO_DATE()` restituisce NULL e genera un *warning*.

Conversione della stringa in data, esempi:

```
INSERT INTO studenti (nome, cognome, email, data_nascita)
VALUES ('marco', 'allegri', 'marco.allegri@gmail.com', STR_TO_DATE('05,10,1969', '%d,%m,%Y'));
```

```
INSERT INTO studenti (nome, cognome, email, data_nascita)
VALUES ('marco', 'allegri', 'marco.allegri@gmail.com', STR_TO_DATE('1 February 2017', '%d %M %Y'));
```

```
SELECT STR_TO_DATE(CONCAT_WS(' ','05','10','1969'), '%d,%m,%Y');
```


Calcoli con date e orari

Per sommare un periodo di tempo ad una data o un orario si possono usare le funzioni **ADDDATE()** e **ADDTIME()**.

```
SELECT ADDDATE('2017-03-01',INTERVAL 5 DAY); #restituisce: 2017-03-06
```

```
SELECT ADDDATE('2017-03-01', 5); #restituisce 2017-03-06
```

Come si vede, per sommare alla data cinque giorni si possono usare due espressioni diverse, INTERVAL 5 DAY o semplicemente il numero 5 (potete aggiungere mesi o anni usando MONTH e YEAR)

```
SELECT ADDDATE('2017-03-01',INTERVAL 5 YEAR); restituisce: 2022-03-06
```

Discorso analogo vale per ADDTIME. Ecco direttamente qualche esempio:

```
SELECT ADDTIME('17:25','05:05'); #restituisce: 22:30:00
```

```
SELECT ADDTIME('17:25','00:05:05'); #restituisce: 17:30:05
```

Nel caso di ADDTIME, si può indicare direttamente il lasso di tempo da sommare.

Sottrazione: **SUBDATE()**/DATE_SUB() il cui funzionamento è speculare a **ADDDATE()**/(DATE_ADD):

```
SELECT SUBDATE('2015-03-01',INTERVAL 5 DAY); #restituisce: 2015-02-24
```

Sottrazione ore/minuti/secondi **SUBTIME()**:

```
SELECT SUBTIME(CURTIME(),'03:03');
```

DATEDIFF().

Questa funzione calcola il numero di giorni che intercorrono tra due date.

Questa funzione accetta due argomenti ed il risultato sarà un numero positivo se la prima data è successiva alla seconda, altrimenti il risultato sarà un numero negativo:

```
SELECT DATEDIFF('2017-03-01','2017-02-10');
```

```
SELECT DATEDIFF('2017-01-01','2017-02-10');
```

```
SELECT DATEDIFF(CURDATE(),'2020-04-10');
```

Se volessimo conoscere i giorni trascorsi da una determinata data dovremmo impostare una query tipo:

```
SELECT DATEDIFF(CURDATE(),data) FROM nome_tabella  
WHERE condizioni;
```


TIMESTAMPADD(unità, intervallo, espr_datetime).

Aggiunge l'espressione intera *intervallo* all'argomento *espr_datetime* DATE o DATETIME.

L'argomento *unità* rappresenta l'unità di misura dell'argomento intervallo, che dovrebbe essere uno dei seguenti valori:

MICROSECOND, SECOND, MINUTE, HOUR, DAY, WEEK, MONTH, QUARTER o *YEAR*.

```
SELECT TIMESTAMPADD(MINUTE, 1, '2003-01-02 00:01:00');
```

```
SELECT TIMESTAMPADD(WEEK, 1, '2003-01-02');
```

TIMESTAMPDIFF(unità, espr1, espr2)

Restituisce il risultato di *espr2* - *espr1*, che sono entrambe di tipo DATETIME.

Una espressione potrebbe essere una data e un'altra un datetime; in questo contesto, un valore DATE viene considerato come un DATETIME che ha come orario '00:00:00'.

Il *risultato* è un intero e l'unità di misura è quella specificata dall'argomento *unità*.

I valori ammessi sono gli stessi che si possono usare per la funzione **TIMESTAMPADD()**.

TIMESTAMPDIFF() può essere utilizzato per calcolare l'età:

```
SELECT TIMESTAMPDIFF(MONTH, '2003-02-01', '2003-05-01');
```

```
SELECT TIMESTAMPDIFF(YEAR, '2002-05-01', '2001-01-01');
```

```
SELECT TIMESTAMPDIFF(MINUTE, '2003-02-01', '2003-05-01 12:05:55');
```

Calcolare età

```
SELECT NOME, COGNOME, TIMESTAMPDIFF(YEAR, data_nascita, CURDATE()) AS `Età`  
FROM studenti;
```

Vediamo come sfruttare questo calcolo in fase di UPDATE e di INSERT.

```
UPDATE studenti  
SET eta = TIMESTAMPDIFF(YEAR, data_nascita, CURDATE());
```

```
INSERT INTO studenti  
(nome, cognome, genere, data_nascita, email, eta)  
VALUES ('paperina', 'duck', 'f', '1999-01-01', 'paperina_d@gmail.com',  
TIMESTAMPDIFF(YEAR, data_nascita, CURDATE()));
```

Anche se, attraverso la registrazione della data di nascita, siamo sempre in grado di ricavare l'età corretta.

Giorni al compleanno

```
SELECT nome, cognome, DAYOFYEAR(CURDATE())-DAYOFYEAR(data_nascita) AS Giorni
FROM studenti
ORDER BY Giorni;
```

Posso aggiungere una condizione per intercettare quelli che ricadono nel prossimo mese:

```
SELECT nome, cognome, data_nascita, DAYOFYEAR(CURDATE())-
DAYOFYEAR(data_nascita) AS Giorni
FROM studenti
WHERE (DAYOFYEAR(CURDATE())-DAYOFYEAR(data_nascita)) BETWEEN -31 AND 0;
```

Se uso WHERE l'alias della colonna non viene intercettata (vedi slide successiva)

```
SELECT nome, cognome, (DAYOFYEAR(CURDATE())-DAYOFYEAR(data_nascita)) AS Giorni
FROM studenti
WHERE Giorni BETWEEN -31 AND 0
ORDER BY Giorni;
/* selezione quelli a cui al compleanno mancano meno di 31 giorni */
```

HAVING

WHERE viene utilizzato per selezionare i dati nelle tabelle originali in elaborazione.

HAVING viene utilizzato per filtrare i dati nel set di risultati prodotto dalla query.

Ciò significa che **HAVING** può fare riferimento a valori aggregati (vedere Raggruppamenti) e alias nella clausola SELECT.

Ricordiamo l'ordine delle istruzioni in mysql:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY  
LIMIT
```

```
SELECT nome, cognome, (DAYOFYEAR(CURDATE())-DAYOFYEAR(data_nascita)) AS Giorni  
FROM studenti  
HAVING Giorni BETWEEN -31 AND 0  
ORDER BY Giorni;  
/* seleziono quelli a cui al compleanno mancano meno di 31 giorni */
```

JSON* function

Inserire i dati in formato JSON, per un attributo definito in questo modo, richiede la notazione JSON

```
INSERT INTO articoli(descrizione, specifiche)
VALUES(
    'TV SAMSUNG 32" SMART TV',
    '{
        "marca": "SAMSUNG",
        "pesoKg": "5.12",
        "schermo": "LCD",
        "pollici": 32,
        "uscite": ["HDMI", "USB"]
    }'
);
```

Per gestire il tipo di dati JSON mysql mette a disposizione una serie di funzioni: ne analizzeremo alcune*.

* [Approfondimento](#)

** JSON function: [elenco completo](#)

`JSON_OBJECT( [ key, val [, key, val ] ...] )`

*La funzione `JSON_OBJECT` accetta un elenco di coppie *chiave/valore* `JSON_OBJECT(key1, value1, key2, value2, ... key(n), value(n))` e *restituisce un oggetto JSON*.*

Si verifica un errore se il nome di una chiave dispari è NULL o il numero di argomenti è dispari.

```
INSERT INTO articoli(descrizione, specifiche)
VALUES(
    'TV SONY 32" SMART TV',
    JSON_OBJECT(
        "marca","SONY","pesoKg","6.5","schermo","LED","pollici",32,"
        uscite","HDMI"
    )
);
```


JSON_ARRAY(([val [, val] ...]))

La funzione JSON_ARRAY accetta in argomento una lista di valori e crea una array in formato JSON.

I valori dell'elenco possono essere di diverso tipo (numeri, stringhe, null e booleani)

```
INSERT INTO articoli(descrizione, specifiche)
VALUES(
    'TV PHILIPS 55" SMART TV',
    JSON_OBJECT(
        "marca", "PHILIPS", "pesoKg", "9.5", "schermo", "LED", "pollici", 55
        , "uscite", JSON_ARRAY( 'HDMI', 'RCA', 'USB', 'COAXIAL', 'SCART' )
    )
);
```


`JSON_EXTRACT(json_doc, path[, path] ... )`

Restituisce i dati da un documento JSON, selezionati dalle parti del documento corrispondenti agli argomenti *path*.

Restituisce NULL se un argomento è NULL o nessun percorso individua un valore nel documento.

Si verifica un errore se l'argomento *json_doc* non è un documento JSON valido o qualsiasi argomento *path* non è un'espressione di percorso valida.

Il valore restituito è costituito da tutti i valori che corrispondono agli argomenti *path*.

Se è possibile che tali argomenti possano restituire più valori, i valori corrispondenti vengono inseriti automaticamente in un array, nell'ordine corrispondente ai percorsi che li hanno prodotti.

In caso contrario, il valore restituito è il singolo valore corrispondente.

Gli attributi (key) del documento JSON si possono intercettare in base al percorso (path) all'interno del JSON.

```
SELECT JSON_EXTRACT(column, '$.key')  
FROM tableName;
```

```
SELECT JSON_EXTRACT(specifiche, '$.uscita')  
FROM articoli;
```

Questa funzione ha un alias che ne semplifica la scrittura.

```
SELECT column -> '$.key'  
FROM tableName;
```

```
SELECT specifiche -> '$.uscita'  
FROM articoli;
```

Vediamo l'uso della funzione con gli array.

```
SELECT JSON_EXTRACT('[10, 20, [30, 40]]', '$[1]');--20
```

```
SELECT JSON_EXTRACT('[10, 20, [30, 40]]', '$[0]', '$[1]');--[10, 20]
```

```
SELECT JSON_EXTRACT('[10, 20, [30, 40]]', '$[2]');--[30, 40]  
SELECT JSON_EXTRACT('[10, 20, [30, 40]]', '$[2][1]');--[40]
```

Vediamo l'uso della funzione con gli array nella tabella articolo.

```
SELECT JSON_EXTRACT(specifiche, '$.uscite[2]')  
FROM articoli;
```

```
SELECT specifiche -> '$.uscite[2]'  
FROM articoli;
```


`JSON_SET(json_doc, path, val[, path, val] ... )`

La funzione `JSON_SET` sostituisce i valori esistenti e aggiunge valori inesistenti.

Restituisce `NULL` se un argomento è `NULL` o se il path fornito non individua un oggetto.

Si verifica un errore se l'argomento *json_doc* non è un documento JSON valido.

```
UPDATE articoli
SET specifiche =
    JSON_SET(specifiche,
        '$.marca', 'LG',
        '$.uscite', JSON_ARRAY('HDMI', 'SCART', 'S/PDIF'),
        '$.ingressi', JSON_ARRAY('ETHERNET', 'USB'))
WHERE id = 1;
```

JSON_INSERT(json_doc, path, val[, path, val] ...)

La funzione JSON_INSERT inserisce i valori senza sostituire i valori esistenti.

```
UPDATE articoli
  SET specifiche = JSON_INSERT(specifiche, '$.uscite[2]', 'RGB')
WHERE id = 1;
-- non produce risultato perché la posizione nell'array è occupata
```

```
UPDATE articoli
  SET specifiche = JSON_INSERT(specifiche, '$.uscite[3]', 'RGB')
WHERE id = 1;
-- aggiunge il nuovo elemento
```

`JSON_REPLACE(json_doc, path, val[, path, val] ... )`

La funzione `JSON_REPLACE` sostituisce solo i valori esistenti.

Se la chiave non esiste il documento non viene modificato.

```
UPDATE articoli
SET specifiche = JSON_REPLACE(specifiche, '$.marca', 'SAMSUNG')
WHERE id = 1;
```

```
UPDATE articoli
SET specifiche = JSON_REPLACE(specifiche, '$.marca', 'SHARP', '$.schermo', 'PLASMA')
WHERE id = 1;
```

Vediamo l'uso della funzione per modificare un elemento di un array.

```
UPDATE articoli
SET specifiche = JSON_REPLACE(specifiche, '$.uscite[1]', 'HDMI2')
WHERE id = 1;
```


JSON_REMOVE(json_doc, path[, path] ...)

La funzione JSON_REMOVE elimina i valori in una colonna di tipo JSON.

Se la proprietà non esiste il documento non viene modificato.

```
UPDATE articolo  
  SET specifiche = JSON_REMOVE(specifiche, '$.profondita')  
WHERE id = 1; -- non produce risultato perché la proprietà non esiste
```

```
UPDATE articolo  
  SET specifiche = JSON_REMOVE(specifiche, '$.uscite[1]')  
WHERE id = 1; -- elimina il secondo elemento della proprietà uscite
```

Control Flow Function (funzioni per il controllo del flusso)*:

IF(expr1, expr2, expr3), CASE

Se *expr1* è vera viene usata *expr2*, se *expr1* è falsa viene usata *expr3*

```
SELECT IF ( 1 > 2 , 0 , 1 ); -- 3
```

```
SELECT IF ( 1 < 2 , 'yes' , 'no' ); -- 'yes'
```

Interroghiamo la tabella studente. Se uno studente proviene dalla provincia di Torino lo consideriamo uno studente *in sede*, altrimenti *fuori sede*:

```
SELECT cognome,  
       IF( provincia = 'to', 'sede', 'fuori sede' ) sede  
FROM studenti  
ORDER BY sede DESC, cognome;
```

*per approfondimento: <https://dev.mysql.com/doc/refman/8.0/en/flow-control-functions.html>

CASE

```
CASE value WHEN [compare_value]
THEN result [WHEN [compare_value]
THEN result ...]
[ELSE result] END
```

```
CASE WHEN [condition]
THEN result [WHEN [condition]
THEN result ...][ELSE result] END
```

Esempio

```
SELECT
    provincia,
    CASE provincia
    WHEN 'to' THEN 'Torino'
    WHEN 'at' THEN 'Asti'
    WHEN 'no' THEN 'Novara'
    WHEN 'al' THEN 'Alessandria'
    WHEN 'cn' THEN 'Cuneo'
    ELSE 'Vercelli' END 'Provincia completa'
FROM studenti;
```

```
SELECT
    titolo,
    prezzo,
    CASE
        WHEN prezzo < 5 THEN 'economico'
        WHEN prezzo >= 5 AND prezzo <= 10 THEN 'medio'
        WHEN prezzo > 10 THEN 'caro'
    END valutazione
FROM libri;
```

Esempio

```
SELECT
    cognome,
    CASE WHEN genere = 'f' THEN 'Donna' ELSE 'Uomo' END Genere
from studenti;
```

Esempio

Consideriamo la query della slide relativa alla istruzione **UNION**, utilizzata per suddividere gli studenti sulla base della generazione di appartenenza (**slide 158**), con la funzione CASE potremmo riscriverla così:

```
SELECT cognome, data_nascita `Data di nascita`,  
CASE WHEN year(data_nascita) <= '1980' THEN 'X'  
      WHEN year(data_nascita) > '1980'  
            AND year(data_nascita) < '1997' THEN 'millenials'  
      WHEN year(data_nascita) >= '1997' THEN 'Z'  
      WHEN year(data_nascita) IS NULL THEN 'Manca data nascita'  
END Generazione  
FROM studenti  
ORDER BY Generazione;
```

Giorni al compleanno, se siamo a dicembre:

```
SELECT
    nome,
    cognome,
    data_nascita,
    CASE
        WHEN MONTH(CURDATE()) = 12 THEN
            DAYOFYEAR(CURDATE()) - (DAYOFYEAR(CURDATE()) -
            DAYOFYEAR(data_nascita))
        ELSE DAYOFYEAR(data_nascita) -
            DAYOFYEAR(CURDATE())
    END AS Giorni
FROM
    studenti
HAVING Giorni BETWEEN 0 AND 31 -- 0 AND 7
ORDER BY Giorni;
```


Raggruppamenti

Elenco dei Valori Distinti in MySQL

Per visualizzare i risultati senza ripetizioni, possiamo usare l'istruzione DISTINCT.

DISTINCT su una singola colonna

Se vogliamo ottenere un elenco di valori unici per una sola colonna, possiamo usare:

```
SELECT DISTINCT cognome  
FROM studenti;
```

Questa query restituisce tutti i cognomi distinti presenti nella tabella studenti.

Se un cognome compare più volte, verrà mostrato solo una volta.

DISTINCT su più colonne

Il DISTINCT si applica all'intera combinazione delle colonne specificate.

Ad esempio:

```
SELECT DISTINCT cognome, nome  
FROM studenti  
ORDER BY cognome;
```

In questo caso, vengono eliminate solo le righe in cui **sia il cognome sia il nome** sono identici.

Se due studenti hanno lo stesso nome ma un cognome diverso, entrambe le righe saranno presenti nel risultato.

Utilizzo di WHERE con DISTINCT

Possiamo combinare DISTINCT con la clausola WHERE per filtrare i risultati prima di eliminare i duplicati:

```
SELECT DISTINCT cognome, nome  
FROM studenti  
WHERE cognome LIKE 'V%'  
ORDER BY cognome;
```

Questa query seleziona solo i cognomi che iniziano con la lettera "V", eliminando i duplicati e ordinando il risultato in ordine alfabetico.

GROUP BY

La clausola GROUP BY è un elemento opzionale che può essere utilizzato all'interno di una SELECT.

Il suo scopo è quello di raggruppare i dati in base ai valori di una o più colonne.

Se vogliamo raggruppare i record della tabella studenti in base al cognome, possiamo usare:

```
SELECT cognome  
FROM studenti  
GROUP BY cognome;
```

In questo caso, il risultato sarà lo stesso di una query con DISTINCT:

```
SELECT DISTINCT cognome  
FROM studenti;
```

Entrambe le query restituiscono un elenco di cognomi univoci.

Tuttavia, GROUP BY viene tipicamente utilizzato in combinazione con funzioni di aggregazione (COUNT(), SUM(), AVG(), ecc.), mentre DISTINCT serve solo per rimuovere duplicati.

Raggruppare i risultati di funzioni aggregate

Le potenzialità di **GROUP BY** emergono:

- quando si utilizza una funzione di aggregazione {**COUNT()**, **AVG()**, **MAX()**, **MIN()**, **SUM()**};

Per contare il numero degli studenti raggruppati per genere possiamo scrivere:

```
SELECT genere, COUNT(genere)
FROM studente
GROUP BY genere;
```

Per contare il numero dei libri raggruppati per editore possiamo scrivere:

```
SELECT e.nome, COUNT(*) quanti
FROM libri l
JOIN editori e
ON e.id=l.editore_id GROUP BY e.nome;
```


Per conoscere l'età media degli studenti raggruppati per genere possiamo scrivere:

```
SELECT genere,  
FLOOR(AVG(TIMESTAMPDIFF(YEAR,data_nascita,CURDATE())) 'età media'  
FROM studenti  
GROUP BY genere;
```

Per conoscere il valore dei libri raggruppati per editore possiamo scrivere:

```
SELECT e.nome, sum(prezzo) valore  
FROM libri l, editori e  
WHERE e.id=l.editore_id  
GROUP BY e.nome  
ORDER BY valore;
```

Per conoscere per ciascun editore quanti libri abbiamo a catalogo, il loro valore, il loro prezzo medio, il più economico e il più caro possiamo scrivere:

```
SELECT
  e.nome,
  count(*) quanti,
  sum(prezzo) valore,
  round(avg(prezzo),2) 'prezzo medio',
  min(prezzo) 'meno caro',
  max(prezzo) 'più caro'
FROM libri l
JOIN editori e
ON e.id=l.editore_id
GROUP BY e.nome
ORDER BY quanti;
```

- quando è utilizzata insieme ad **HAVING** che aggiunge un filtro su valori raggruppati:

```
SELECT
  e.nome,
  count(*) quanti,
  sum(prezzo) valore,
  round(avg(prezzo),2) 'prezzo medio',
  min(prezzo) 'meno caro',
  max(prezzo) 'più caro'
FROM libri l
JOIN editori e
ON e.id=l.editore_id
GROUP BY e.nome
HAVING quanti > 1
ORDER BY quanti;
```


- quando è utilizzata insieme ad **HAVING** che aggiunge un filtro su valori raggruppati:

```
SELECT cognome, COUNT(cognome) AS 'numero'  
FROM studenti  
GROUP BY cognome  
HAVING numero > 1  
ORDER BY cognome;
```

Mediante questa query abbiamo filtrato il resultset mostrando unicamente quei cognomi per i quali è possibile rinvenire più di una occorrenza all'interno della tabella.

```
SELECT provincia, genere, COUNT(*) AS 'numero'  
FROM studenti  
GROUP BY provincia, genere  
HAVING numero > 1  
ORDER BY provincia, genere;
```

Mediante questa query abbiamo raggruppato per provincia e genere gli studenti, contandone il numero; la seconda istruzione filtra ulteriormente il gruppo considerando i valori contati maggiori di 1.

```
SELECT genere, COUNT(cognome) numero
FROM studenti
WHERE provincia = 'to'
GROUP BY genere
HAVING numero > 1
ORDER BY genere DESC;
```

WHERE può essere usato per ridurre il result set usato ma prima del GROUP BY:

WHERE filtra le righe;

per un filtro ulteriore *sul gruppo* usate **HAVING**.

```
SELECT
    provincia,
    genere,
    FLOOR(AVG(TIMESTAMPDIFF(YEAR, data_nascita, curdate())) 'età media',
    count(*) numero
FROM studenti
GROUP BY provincia, genere
HAVING numero > 1
ORDER BY provincia, genere
```

GROUP BY ... WITH ROLLUP (istruzione non standard SQL):

```
SELECT provincia, count(*) AS 'numero'  
FROM studenti  
GROUP BY provincia WITH ROLLUP;
```

Mediante questa query abbiamo contato gli studenti divisi per provincia.

Con l'aggiunta dell'istruzione (non SQL) WITH ROLLUP siamo in grado di produrre una riga in più con il totale

```
SELECT provincia, genere, count(*) AS 'numero'  
FROM studenti  
GROUP BY provincia, genere WITH ROLLUP;
```

Mediante questa query abbiamo contato gli studenti divisi per provincia e per genere.

Con l'aggiunta dell'istruzione (non SQL) WITH ROLLUP siamo in grado di produrre le righe in più con i sub totali e il totale.

GROUP BY ... WITH ROLLUP e GROUPING

Con l'aggiunta dell'istruzione WITH ROLLUP si produce una riga in più con i subtotali delle province, il valore del superaggregato province è **NULL**

Per aggiungere un'etichetta e non riportare il valore NULL possiamo usare la funzione `grouping()` che distingue le righe del superaggregato dalle righe regolari.

```
SELECT
    provincia,
    count(*) AS 'numero',
    grouping(provincia)
FROM studenti
GROUP BY provincia WITH ROLLUP;
```

GROUPING restituisce 0 per le righe regolari e 1 per il superaggregato.

Con l'istruzione IF possiamo quindi assegnare un'etichetta al superaggregato:

```
SELECT
    IF(GROUPING(provincia), 'tutte le province', provincia) AS provincia,
    count(*) AS 'numero'
FROM studenti
GROUP BY provincia WITH ROLLUP;
```

```
SELECT
    provincia,
    genere,
    count(*) AS 'numero',
    grouping(provincia),
    grouping(genere)
FROM studenti
GROUP BY provincia, genere WITH ROLLUP;
```

```
SELECT
    IF(GROUPING(provincia), 'totale province', provincia) AS
provincia,
    IF(GROUPING(genere), 'totale generi', genere) AS genere,
    count(*) AS 'numero'
FROM studenti
GROUP BY provincia, genere WITH ROLLUP;
```